

Diseño y desarrollo de un sistema de videollamada accesible con reconocimiento de lengua de signos española y síntesis de voz

Pablo Colomino Granell

Tutor: Jorge Igual García

Junio de 2026

Resumen

Este proyecto presenta el diseño y desarrollo de un sistema de videollamada accesible orientado a personas sordas, con dificultades auditivas o con limitaciones en la expresión oral. El sistema emplea visión artificial para extraer en tiempo real los puntos clave de las manos y una red neuronal BiLSTM entrenada con un dataset propio de Lengua de Signos Española (LSE) para clasificar los gestos. Las glosas reconocidas se acumulan y se traducen a español natural mediante un modelo de lenguaje local, sintetizando el resultado en voz. El proyecto abarca desde la captura del dataset hasta el despliegue completo en una aplicación web en tiempo real.

Resum

Este projecte presenta el disseny i desenvolupament d'un sistema de videotelefonada accessible orientat a persones sordes, amb dificultats auditives o amb limitacions en l'expressió oral. El sistema empra visió artificial per a extraure en temps real els punts clau de les mans, i una xarxa neuronal entrenada amb un dataset propi de Llengua de Signes Espanyola (LSE) per a classificar els gestos. Les gloses reconegudes s'acumulen i es tradueixen a espanyol natural mitjançant un model de llenguatge local, sintetitzant el resultat en veu. El projecte abasta des de la captura del dataset fins al desplegament complet en una aplicació web en temps real.

Abstract

This project presents the design and development of an accessible video call system aimed at people who are deaf, have hearing difficulties or have limitations in spoken expression. The system uses computer vision to extract key hand features in real time, and a neural network trained on a proprietary dataset of Spanish Sign Language (LSE) to classify gestures. The recognised annotations are aggregated and translated into natural Spanish using a local language model, with the result synthesised into speech. The project spans the entire process, from dataset collection to full deployment in a real-time web application.



RESUMEN EJECUTIVO

La memoria del TFG del GTDM debe desarrollar en el texto los siguientes conceptos, debidamente justificados y discutidos, centrados en el ámbito de la TDM

CONCEPT (ABET)	CONCEPTO (traducción)	¿Cumple? (S/N)	¿Dónde? (páginas)
1. IDENTIFY:	1. IDENTIFICAR:		
1.1. Problem statement and opportunity	1.1. Planteamiento del problema y oportunidad	S	1
1.2. Constraints (standards, codes, needs, requirements & specifications)	1.2. Toma en consideración de los condicionantes (normas técnicas y regulación, necesidades, requisitos y especificaciones)	S	2
1.3. Setting of goals	1.3. Establecimiento de objetivos	S	5
2. FORMULATE:	2. FORMULAR:		
2.1. Creative solution generation (analysis)	2.1. Generación de soluciones creativas (análisis)	S	8
2.2. Evaluation of multiple solutions and decision-making (synthesis)	2.2. Evaluación de múltiples soluciones y toma de decisiones (síntesis)	S	10
3. SOLVE:	3. RESOLVER:		
3.1. Fulfilment of goals	3.1. Evaluación del cumplimiento de objetivos	S	46
3.2. Overall impact and significance (contributions and practical recommendations)	3.2. Evaluación del impacto global y alcance (contribuciones y recomendaciones prácticas)	S	45

Índice

I Memoria

1	Introducción	1
1.1	Contexto y motivación	1
1.2	Solución propuesta	1
1.3	Condicionantes del proyecto	2
1.4	Relación del Proyecto con los Estudios del Grado	2
1.5	Estructura del documento	3
2	Objetivos	5
2.1	Objetivo general	5
2.2	Objetivos específicos	5
3	Estado del arte	7
3.1	La Lengua de Signos Española	7
3.2	Reconocimiento automático de lengua de signos	7
3.3	MediaPipe Hands	8
3.3.1	Análisis Comparativo de Frameworks de Estimación de Pose	8
3.4	Tecnologías de Comunicación Multimedia en la Web	10
3.4.1	El protocolo WebSocket y la comunicación Full-Duplex	10
3.4.2	La arquitectura WebRTC y los mecanismos de señalización NAT	11
3.4.3	Análisis comparativo para pipelines de procesamiento de IA	12
3.5	Redes LSTM y BiLSTM	12
3.6	Modelos de lenguaje y traducción automática	14
4	Metodología	17
4.1	Diseño general del sistema y arquitectura de red	17
4.2	Adquisición y creación del dataset propio	19
4.2.1	Selección del conjunto de signos y clase de control	19
4.2.2	Protocolo de captura de datos	20
4.2.3	Herramienta de Captura y Segmentación Semiautomática	20
4.3	Procesamiento de datos y extracción de características	21
4.3.1	Mapeo espacial de landmarks e indexación simétrica	21
4.3.2	Técnicas de aumentación de datos (Data Augmentation)	23
4.3.3	Estrategia de Entrenamiento e Hiperparámetros	24
4.3.4	Arquitectura y Configuración del Clasificador BiLSTM	24
4.3.5	Auditoría Metodológica y Prevención de Filtración de Datos (Data Leakage)	25
5	Desarrollo e Implementación del Sistema	27

5.1	Implementación del Servidor Backend (Flask y Socket.IO)	27
5.2	Desarrollo de la Interfaz de Usuario (Frontend en React)	28
5.2.1	Distribución de Componentes y Diseño de Experiencia de Usuario (UX)	29
5.2.1.1	Lienzo Multimedia y Botones de Control Local	29
5.2.1.2	Módulo Integrado de Subtitulación y Acumulación de Glosas	30
5.2.1.3	Panel Lateral de Configuración (Sidebar)	30
5.2.2	Integración de Comunicación Bidireccional mediante Speech-to-Text (STT)	31
5.3	Integración Local del Motor de Traducción (Ollama / Llama 3.2)	32
5.4	Especificación del Protocolo de Mensajes e Intercambio de Datos	33
5.4.1	Flujo de Subida: Evento <code>video_frame</code>	33
5.4.2	Flujo de Bajada: Evento <code>prediction_result</code>	33
5.4.3	Flujo de Control de Buffer: Evento <code>word_added</code>	34
5.4.4	Flujo de Salida Semántica: Evento <code>translation_result</code>	34
6	Resultados	37
6.1	Resultados del Entrenamiento del Modelo BiLSTM	37
6.1.1	Análisis de la Matriz de Confusión y Validación del Rendimiento	39
6.2	Evaluación de Latencia y Rendimiento en Tiempo Real	40
7	Pliego de Condiciones y Presupuesto	41
7.1	Pliego de Condiciones Técnicas	41
7.1.1	Especificaciones de Hardware y Equipos Mínimos	41
7.1.2	Entorno de Software y Dependencias de Ejecución	42
7.1.3	Condiciones Legales, Privacidad y Licenciamiento	42
7.2	Presupuesto General del Proyecto	43
7.2.1	Costes Directos de Personal	43
7.2.2	Planificación Temporal del Proyecto	43
7.2.3	Costes Indirectos: Amortización de Equipamiento	43
7.2.4	Costes de Consumo de Servicios de Terceros	44
7.2.5	Resumen del Presupuesto Económico General	44
8	Conclusiones y trabajo futuro	45
8.1	Conclusiones Generales	45
8.2	Obtención de Objetivos	46
8.3	Líneas Futuras	46
	Bibliografía	49
	Bibliografía	49
II	Anexos	
A	Manual de Instalación y Configuración del Sistema	53
A.1	Requisitos Previos del Sistema	53
A.2	Configuración del Entorno de Ejecución Local (Ollama)	53
A.3	Instalación y Despliegue del Servidor Backend	54
A.4	Instalación y Despliegue de la Interfaz de Usuario (Frontend)	55

Índice de figuras

3.1	Puntos clave de la mano extraídos por MediaPipe Hands.	8
3.2	Arquitectura interna de una celda de memoria LSTM convencional con el flujo del estado de la celda (c_t) y sus compuertas de control.	13
3.3	Flujo de procesamiento bidireccional en una capa BiLSTM mediante la concatenación de las capas Forward y Backward.	14
3.4	Arquitectura del flujo de procesamiento de lenguaje natural (NLP) local para la corrección gramatical de secuencias gestuales.	15
4.1	Diagrama de bloques conceptual del pipeline del sistema, mostrando el flujo de datos bidireccional en tiempo real. El bloque de síntesis de voz (Google Cloud TTS) es el único componente que requiere conectividad a internet; el resto del pipeline de inteligencia artificial se ejecuta de forma local.	18
4.2	Interfaz gráfica del entorno de desarrollo de captura durante la adquisición de datos en tiempo real.	22
5.1	Visualización de la interfaz gráfica de la aplicación	29
6.1	Evolución de la precisión y función de pérdida durante el entrenamiento del modelo BiLSTM.	38
6.2	Matriz de confusión en valores absolutos para el clasificador BiLSTM de 13 clases.	39

Índice de tablas

3.1	Análisis comparativo de frameworks de estimación de pose y seguimiento dactilar enfocado a entornos web interactivos.	9
3.2	Análisis comparativo entre WebSockets y WebRTC enfocado al procesamiento de IA y visión artificial en tiempo real.	12
3.3	Ejemplos prácticos de reestructuración sintáctica y traducción de LSE a castellano natural mediante el LLM local.	15
4.1	Vocabulario completo del dataset propio de LSE entrenado en el sistema.	19
6.1	Resumen de métricas de rendimiento del sistema de reconocimiento de LSE.	38
7.1	Desglose detallado de los costes directos de personal técnico de ingeniería.	43
7.2	Desglose temporal de las horas de dedicación por fase del proyecto.	43
7.3	Cálculo de la amortización lineal de la infraestructura de hardware empleada.	44
7.4	Costes de licencias de software y consumo de APIs de infraestructura en la nube.	44
7.5	Resumen consolidado del presupuesto económico total de ingeniería del proyecto.	44

Relación del trabajo con los ODS

Objetivo de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza				X
ODS 2. Hambre cero				X
ODS 3. Salud y bienestar				X
ODS 4. Educación de calidad	X			
ODS 5. Igualdad de género				X
ODS 6. Agua limpia y saneamiento				X
ODS 7. Energía asequible				X
ODS 8. Trabajo decente y crecimiento económico		X		
ODS 9. Industria, innovación e infraestructuras		X		
ODS 10. Reducción de las desigualdades	X			
ODS 11. Ciudades y comunidades sostenibles				X
ODS 12. Producción y consumo responsables				X
ODS 13. Acción por el clima				X
ODS 14. Vida submarina				X
ODS 15. Vida de ecosistemas terrestres				X
ODS 16. Paz, justicia e instituciones sólidas				X
ODS 17. Alianzas para lograr objetivos				X

Alineación con los ODS

ODS 4 — Educación de calidad. El sistema desarrollado contribuye a eliminar barreras de comunicación en el ámbito educativo, facilitando la participación de personas sordas en entornos de aprendizaje donde la comunicación oral es predominante.

ODS 10 — Reducción de las desigualdades. La herramienta desarrollada reduce directamente la brecha de accesibilidad que existe entre personas sordas y oyentes en el contexto de la comunicación digital, promoviendo la inclusión social de un colectivo históricamente en desventaja.

ODS 8 - Trabajo decente y crecimiento económico: El proyecto facilita la inclusión laboral de personas en procesos de selección y dinámicas de trabajo diarias, promoviendo el derecho a un empleo pleno, productivo y en igualdad de condiciones.

ODS 9 - Industria, innovación e infraestructuras: El proyecto constituye un avance en innovación aplicada al desarrollo de software asistivo mediante el despliegue de clasificadores neuronales (*BiLSTM*) acoplados a modelos de lenguaje (*Llama 3.2*).

Parte I

Memoria

Capítulo 1

Introducción

1.1 Contexto y motivación

La comunicación es una necesidad fundamental del ser humano. Sin embargo, para las personas con dificultades auditivas o del habla, la comunicación representa una barrera significativa en su vida cotidiana. En España, se estima que entre 70.000 y 150.000 personas se comunican con lengua de signos [1], lo que pone de manifiesto la necesidad de desarrollar herramientas tecnológicas que faciliten su inclusión en la sociedad.

El auge de las videollamadas ha transformado la manera en que las personas se comunican tanto en el ámbito personal como profesional. No obstante, estas plataformas no están adaptadas para facilitar la comunicación entre personas oyentes y personas con diversidad funcional auditivo-vocal, lo que genera una brecha de accesibilidad importante.

Los avances recientes en visión artificial ofrecen una oportunidad única para abordar este problema. Tecnologías como MediaPipe, desarrollada por Google, permiten extraer en tiempo real los puntos clave de las manos con gran precisión, mientras que las redes neuronales recurrentes, especialmente las arquitecturas LSTM, han demostrado ser eficaces en la clasificación de secuencias temporales como los gestos de la lengua de signos.

1.2 Solución propuesta

En este trabajo se presenta el diseño y desarrollo de un sistema de videollamada accesible que integra reconocimiento automático de LSE y síntesis de voz. El sistema es capaz de detectar y clasificar gestos de la LSE en tiempo real a partir de la imagen de la cámara del usuario, mostrando el texto correspondiente al signo reconocido y sintetizándolo en voz para el resto de participantes de la videollamada.

El sistema diseñado se compone en base a tres pilares fundamentales: un *frontend* interactivo en React, un *backend* de alta disponibilidad en Python y un motor de computación inteligente modular. No obstante, para garantizar una traducción bidireccional fluida y accesible, este núcleo se subdivide en tres sub-sistemas especializados que operan de forma secuencial y síncrona:

1. **Módulo de Extracción y Clasificación Secuencial (MediaPipe + BiLSTM):** Es el encar-

gado del procesamiento de vídeo en tiempo real a nivel de fotograma. Aísla las coordenadas geométricas de las manos y clasifica las trayectorias temporales transformándolas de forma asíncrona en unidades léxicas o *glosas* de la LSE.

2. **Motor de Traducción y Reestructuración Semántica (Ollama / Llama 3.2):** Actúa como la capa de procesamiento del lenguaje natural. Su función es crítica, ya que la gramática de la LSE difiere formalmente del castellano (carece de nexos, artículos y tiempos verbales conjugados en la estructura del signo). Este módulo local reordena las glosas entrantes y lo traduce a oraciones sintácticamente correctas y naturales.
3. **Subsistema de Síntesis Multimedia Bidireccional (TTS / STT):** Representa la capa de accesibilidad universal e interacción persona-computador. Por un lado, transforma la oración de texto generada por la IA en un flujo de audio mediante tecnologías de síntesis de voz (*Text-to-Speech*), permitiendo la expresión del usuario signante. Por otro lado, implementa la transcripción de voz a texto (*Speech-to-Text*) para subtitular en tiempo real las respuestas del interlocutor oyente.

1.3 Condicionantes del proyecto

El diseño de la solución propuesta se encuentra enmarcado por una serie de condicionantes normativos, técnicos y funcionales que han guiado las decisiones de ingeniería adoptadas. En el plano normativo, la *Ley 27/2007*, de 23 de octubre [2], reconoce oficialmente las lenguas de signos españolas y regula los medios de apoyo a la comunicación de las personas sordas, constituyendo el marco legal de referencia para cualquier herramienta tecnológica orientada a este colectivo. En el plano técnico, la elección de los protocolos de comunicación en tiempo real debe ajustarse a estándares abiertos e interoperables, como el protocolo WebSocket estandarizado en la *RFC 6455* [3], que garantiza la compatibilidad del sistema con cualquier navegador web moderno sin necesidad de complementos propietarios. Finalmente, en el plano funcional, el sistema debe responder a las necesidades reales del colectivo sordo y con dificultades del habla, evitando introducir barreras adicionales de accesibilidad en ámbitos como la sanidad, la educación o el entorno digital [4], y operando bajo restricciones de baja latencia y ejecución local que aseguren la viabilidad práctica de una comunicación fluida en tiempo real.

1.4 Relación del Proyecto con los Estudios del Grado

El desarrollo de este Trabajo de Fin de Grado representa un acercamiento práctico a las competencias transversales y específicas adquiridas a lo largo del *Grado en Tecnología Digital y Multimedia*. Lejos de limitarse a una única disciplina, el sistema de comunicación accesible requiere un enfoque multidisciplinar que unifica cuatro de los grandes itinerarios formativos de la titulación: el desarrollo de sistemas web de alta eficiencia, el procesamiento de señales multimedia, la integración de sistemas basados en inteligencia artificial y la planificación y gestión económica de proyectos tecnológicos.

A continuación, se desglosa cómo las diferentes áreas de conocimiento del grado se han articulado de forma sinérgica en el proyecto:

- **Desarrollo de Aplicaciones Web y Gestión de Flujos de Datos:** La implementación del *frontend* mediante la biblioteca React y la gestión del estado dinámico del sistema reflejan los principios de la ingeniería del *software* web estudiados en el grado. Asimismo, la arquitectura de comunicación basada en WebSockets (*Socket.IO*) y los mecanismos de control de flujo mediante referencias síncronas demuestran la integración de conceptos clave sobre redes, protocolos de comunicación y optimización del rendimiento en aplicaciones.
- **Ingeniería Multimedia y Procesamiento de Señal (Audio y Vídeo):** El núcleo del sistema interactúa directamente con periféricos de captura local mediante la API `getUserMedia`. El proyecto ha requerido la manipulación multimedia en tiempo real mediante el renderizado matricial en lienzos gráficos, la superposición de mallas de realidad aumentada y la coordinación síncrona de flujos de audio y voz a través de tecnologías de síntesis (*Text-to-Speech*) y transcripción de voz (*Speech-to-Text*).
- **Sistemas Inteligentes e Interacción Persona-Computador:** El uso de visión artificial, junto con el despliegue de redes neuronales recurrentes BiLSTM y modelos de lenguaje locales (Llama 3.2 vía Ollama), materializa las competencias en el ámbito de la inteligencia artificial y el análisis de datos orientados a la accesibilidad.
- **Diseño de Interfaces (UI/UX) y Accesibilidad Universal:** Uno de los pilares del Grado es el diseño centrado en el usuario. Este proyecto justifica dicha competencia mediante la maquetación responsiva con Tailwind CSS y la creación de una interfaz adaptada para usuarios con diversidad funcional auditiva y verbal.
- **Organización y Dirección de Proyectos Tecnológicos:** Las competencias adquiridas en las asignaturas de Organización Digital y Dirección de Proyectos se materializan en la planificación temporal del desarrollo por fases y en la elaboración del presupuesto económico del prototipo (Pliego de Condiciones Técnicas), incluyendo la estimación de costes de personal, la amortización de equipamiento y el análisis de los servicios de terceros consumidos.

En conclusión, este proyecto no solo valida la capacidad técnica para construir una arquitectura de *software* compleja, sino que demuestra la madurez como Ingeniero Multimedia al ser capaz de unificar hardware, programación, diseño de interacción y modelos de computación inteligente para resolver un problema de alto impacto social.

1.5 Estructura del documento

El presente documento se organiza de la siguiente manera. El Capítulo 2 define los objetivos del proyecto. El Capítulo 3 revisa el estado del arte en reconocimiento de lengua de signos y tecnologías relacionadas. El Capítulo 4 describe la metodología seguida para la captura de datos y el entrenamiento del modelo. El Capítulo 5 detalla el desarrollo del sistema completo. El Capítulo 6 presenta y analiza los resultados obtenidos. El Capítulo 7 establece el pliego de condiciones técnicas y el presupuesto general del desarrollo. El Capítulo 8 recoge las conclusiones del trabajo y propone líneas de trabajo futuro. Finalmente, se incluye el Anexo A, el cual constituye un manual detallado de instalación, configuración y despliegue local de la arquitectura multimedia diseñada.

Capítulo 2

Objetivos

2.1 Objetivo general

El objetivo principal de este trabajo es diseñar y desarrollar un sistema de videollamada accesible que permita la comunicación en tiempo real entre personas oyentes y personas con dificultades auditivas o de expresión oral, eliminando la barrera de comunicación existente mediante el reconocimiento automático de la Lengua de Signos Española (LSE) y la síntesis de voz.

2.2 Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. Crear un dataset propio de secuencias de gestos de la LSE, capturando muestras de cada signo mediante la extracción de puntos clave de las manos con MediaPipe.
2. Diseñar y entrenar un modelo de red neuronal BiLSTM capaz de clasificar gestos de la LSE en tiempo real a partir de secuencias temporales de puntos clave de las manos.
3. Desarrollar un servidor en Python con Flask y Socket.IO que gestione el procesamiento de los frames de vídeo en tiempo real y sirva las predicciones del modelo a los clientes conectados.
4. Implementar una interfaz web en React que integre la funcionalidad de videollamada con la capa de reconocimiento de signos, mostrando el texto reconocido a los participantes de la llamada.
5. Validar el sistema completo mediante pruebas de reconocimiento, evaluando la precisión del modelo y la latencia del sistema en condiciones reales de uso.

Capítulo 3

Estado del arte

3.1 La Lengua de Signos Española

La Lengua de Signos Española (LSE) es una lengua compleja con gramática y estructura propias, utilizada principalmente por la comunidad sorda en España, así como por personas con limitaciones severas en la expresión oral que encuentran en este canal gestual su principal vía de comunicación. A diferencia de lo que comúnmente se cree, no es una representación gestual del español oral, sino una lengua independiente con sus propias reglas morfológicas y sintácticas.

En España, su reconocimiento oficial llegó con la *Ley 27/2007*, de 23 de octubre [2], por la que se reconocen las lenguas de signos españolas y se regulan los medios de apoyo a la comunicación oral de las personas sordas, con discapacidad auditiva y sordociegas. A pesar de este respaldo legal, las personas sordas y aquellos usuarios con diversidad funcional del habla siguen encontrando barreras de accesibilidad en ámbitos como la sanidad, la educación o el entorno digital [4] donde la falta de herramientas bidireccionales en tiempo real perpetúa su aislamiento comunicativo.

3.2 Reconocimiento automático de lengua de signos

El reconocimiento automático de lengua de signos (en inglés, *Sign Language Recognition*, SLR) es un campo de investigación activo dentro de la visión artificial y el aprendizaje automático. [5] Su objetivo es desarrollar sistemas capaces de interpretar y traducir gestos de lengua de signos de forma automática.

Los primeros enfoques del SLR se basaban en sensores físicos como guantes instrumentados o dispositivos de captura de movimiento, que aunque ofrecían alta precisión, resultaban intrusivos e imprácticos para un uso cotidiano. Con el avance de las cámaras y las técnicas de visión por computador, los sistemas basados en vídeo se convirtieron en el enfoque dominante, ofreciendo una experiencia mucho más natural.

Dentro de los sistemas basados en vídeo, se distinguen dos grandes categorías: el reconocimiento de gestos estáticos, que clasifica configuraciones de la mano en un único fotograma, y el reconocimiento de gestos dinámicos, que analiza secuencias temporales de movimiento. La LSE, al ser una lengua con componentes dinámicos esenciales, requiere necesariamente un enfoque de

reconocimiento temporal.

3.3 MediaPipe Hands

MediaPipe es un framework de código abierto desarrollado por Google que proporciona soluciones de aprendizaje automático para el procesamiento de datos multimedia en tiempo real. Entre sus módulos, MediaPipe Hands permite la detección y el seguimiento de las manos a partir de imágenes o vídeo, extrayendo los 21 puntos clave o *landmarks* de cada mano con sus coordenadas tridimensionales (x, y, z).

Cada mano se representa mediante 21 puntos que cubren las articulaciones de los cinco dedos y la palma, proporcionando una descripción geométrica completa de la configuración de la mano en cada fotograma. Esta representación es compacta, ligera y suficientemente descriptiva para capturar los gestos de la LSE, lo que la hace especialmente adecuada para aplicaciones en tiempo real.

Una de las principales ventajas de MediaPipe frente a otros enfoques basados en imagen completa es que al trabajar con coordenadas numéricas en lugar de píxeles, el sistema es robusto frente a variaciones de iluminación, color de piel o fondo de la imagen. Además, al ser coordenadas normalizadas entre 0 y 1 respecto al tamaño del fotograma, el sistema es independiente de la resolución de la cámara utilizada.

En el presente trabajo, MediaPipe Hands se utiliza para extraer en tiempo real los puntos clave de ambas manos, generando por cada fotograma un vector de 126 valores que representa las coordenadas x, y, z de los 21 puntos de cada mano ($21 \text{ puntos} \times 3 \text{ coordenadas} \times 2 \text{ manos} = 126 \text{ valores}$), tal y como se muestra en la Figura 3.1. Este vector constituye la entrada al modelo de clasificación BiLSTM.

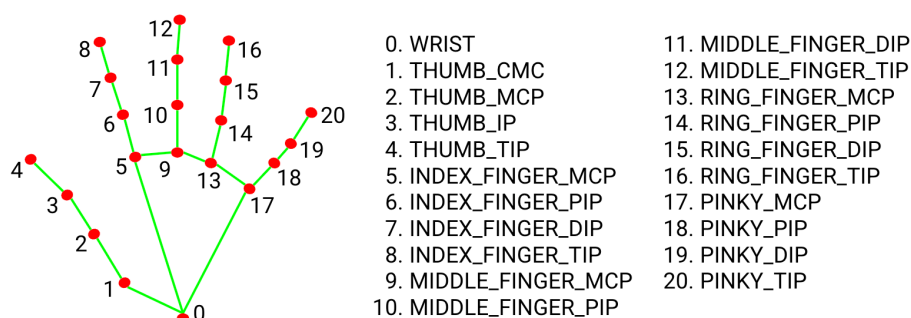


Figura 3.1: Puntos clave de la mano extraídos por MediaPipe Hands.

3.3.1 Análisis Comparativo de Frameworks de Estimación de Pose

La selección del entorno informático para la extracción de características (*features*) es un factor determinante en el diseño de sistemas de visión artificial en tiempo real. En el ámbito de la estimación de pose y el seguimiento dactilar (*hand tracking*), la literatura científica actual ofrece diferentes soluciones basadas en aprendizaje profundo. Con el propósito de fundamentar técni-

camente la elección de *MediaPipe Hands* en este Trabajo de Fin de Grado, se ha realizado un análisis comparativo frente a las dos alternativas metodológicas dominantes en el estado del arte: *OpenPose* y los modelos basados en la arquitectura YOLO (*YOLO-Pose*).

A continuación, se desglosan las características operativas de cada una de estas tecnologías:

- **OpenPose (CMU Graphics Lab):** Es considerado uno de los pioneros en la estimación de pose *bottom-up*, permitiendo la detección conjunta de cuerpo, rostro y manos de múltiples personas. Aunque su precisión espacial es excelente debido al uso de campos de afinidad por partes (*Part Affinity Fields*, PAFs), presenta un gran coste computacional. Requiere de forma obligatoria unidades de procesamiento gráfico dedicadas (GPU de altas prestaciones) para la ejecución de sus convoluciones, lo que invalida por completo su despliegue en entornos cliente web ligeros.
- **YOLO-Pose (You Only Look Once):** Basado en detectores de una sola etapa (*single-shot*), integra la estimación de puntos clave directamente en la tubería de regresión de cajas delimitadoras (*bounding boxes*). Sin embargo, su arquitectura está optimizada para la detección del esqueleto corporal completo (*body pose*), requiriendo un proceso complejo de reentrenamiento (*fine-tuning*) y conjuntos de datos masivos si se desea aislar de forma exclusiva y con alta resolución la topología fina de las articulaciones dactilares.
- **MediaPipe Hands (Google Edge Vision):** Implementa una arquitectura en dos etapas basada en un detector de palmas (*BlazePalm*) y un modelo de puntos clave optimizado para operar sobre regiones de interés (ROI). A diferencia de los enfoques anteriores, este paradigma permite inferir las coordenadas tridimensionales normalizadas de 21 *landmarks* por mano con un consumo de recursos mínimo, permitiendo su ejecución nativa en la CPU de dispositivos de consumo ordinarios.

Para estructurar de manera sistemática este compromiso de ingeniería (*trade-off*), la Tabla 3.1 sintetiza los criterios de rendimiento, carga computacional y viabilidad de integración web de los tres entornos evaluados.

Criterio Técnico	OpenPose	YOLO-Pose	MediaPipe Hands
Enfoque de Red	<i>Bottom-Up</i> (Detección global y ensamblado).	<i>Single-Shot</i> (Regresión directa de puntos).	<i>Top-Down</i> (Detector de palma + ROI local).
Hardware Requerido	GPU dedicada obligatoria.	Requiere aceleración gráfica para baja latencia.	Optimizado para CPU convencionales.
Latencia en Servidor	Elevada (> 50 ms sin hardware masivo).	Media (15 – 30 ms según escala del modelo).	Ultra-baja (< 10 ms por frame en CPU).
Integración en Navegador Web	Inviabile (Requiere dependencias de C++ nativas).	Compleja (Requiere conversión pesada a ONNX/WASM).	Nativa y ligera (Librerías optimizadas en JS/WASM).
Granularidad en Manos	Alta (Requiere activar módulos adicionales).	Baja (Enfoque centrado en articulaciones mayores).	Excelente (21 puntos clave tridimensionales dedicados).

Tabla 3.1: Análisis comparativo de frameworks de estimación de pose y seguimiento dactilar enfocado a entornos web interactivos.

A este respecto, es importante fundamentar el impacto que la elección del framework ejerce sobre la arquitectura física del sistema, específicamente en la división entre la Unidad Central de Procesamiento (CPU) y la Unidad de Procesamiento Gráfico (GPU). Los modelos convencionales de aprendizaje profundo aplicados a la visión artificial, tales como OpenPose o las variantes estándar de YOLO, operan mediante convoluciones masivas sobre matrices de píxeles en bruto. Esta naturaleza matemática requiere un procesamiento en paralelo masivo que solo las GPUs de altas prestaciones pueden solventar en tiempos compatibles con el tiempo real. Sin embargo, delegar la viabilidad de esta herramienta a la disponibilidad de una GPU dedicada penaliza directamente los criterios de accesibilidad universal y bajo coste, limitando su uso a dispositivos de gama alta o encareciendo el despliegue con infraestructura comercial en la nube.

La ventaja competitiva de la arquitectura implementada en este proyecto radica en romper dicha dependencia tecnológica mediante un cambio de paradigma en el flujo de datos. Al emplear *MediaPipe Hands*, el flujo masivo de vídeo se abstrae de forma inmediata en un vector compacto de únicamente 126 características numéricas continuas por fotograma. Operar sobre coordenadas geométricas tridimensionales en lugar de matrices densas de píxeles reduce la carga computacional en varios órdenes de magnitud. Como consecuencia directa, tanto la extracción cinemática como la inferencia secuencial del clasificador BiLSTM pueden ser resueltas de forma nativa por una CPU convencional en una ventana temporal inferior a los 15 ms por cuadro. Esta optimización a nivel de software extiende el acceso a la plataforma, garantizando que cualquier usuario final pueda establecer una videollamada accesible y fluida empleando hardware ordinario, sin necesidad de aceleración gráfica dedicada ni dependencias de red externas.

Por lo tanto, la justificación de diseño de este proyecto se sustenta en los datos expuestos. Dado que el objetivo es construir un sistema de videollamada accesible integrado en una interfaz web reactiva, delegar la extracción cinemática a un framework como *MediaPipe* aporta dos ventajas estratégicas. En primer lugar, generaliza el acceso al software, garantizando que el usuario final no requiera ordenadores con tarjetas gráficas dedicadas para poder comunicarse. En segundo lugar, al transformar el flujo de píxeles del vídeo en una matriz compacta de 126 flotantes desde el inicio del pipeline, se optimiza de forma drástica el ancho de banda del canal de WebSockets, enviando estructuras numéricas puras al servidor de Flask en lugar de secuencias multimedia masivas, lo que minimiza la latencia global del sistema.

3.4 Tecnologías de Comunicación Multimedia en la Web

El desarrollo de sistemas interactivos basados en flujos multimedia en tiempo real exige el análisis de los protocolos de red encargados de garantizar transmisiones de baja latencia. En el ecosistema actual, la evolución tecnológica ha desplazado los antiguos sistemas basados en complementos de terceros hacia arquitecturas nativas integradas directamente en los navegadores, destacando el uso de *WebSockets* y la pila tecnológica *WebRTC*.

3.4.1 El protocolo WebSocket y la comunicación Full-Duplex

El protocolo WebSocket, estandarizado por la IETF en la *RFC 6455* [3], define un canal de comunicación persistente, bidireccional y de baja sobrecarga (Full-Duplex) que opera sobre una única conexión TCP. A diferencia del modelo tradicional HTTP, fundamentado en un modelo de

petición-respuesta síncrono, WebSocket permite el intercambio de datos asíncrono e independiente en cualquier dirección.

El ciclo de vida de este protocolo se rige por las siguientes etapas operativas:

- **Handshake HTTP:** El cliente web inicia una petición estándar HTTP exponiendo una cabecera de actualización (`Upgrade: websocket`). Si el servidor backend acepta las condiciones, devuelve un código de estado 101 `Switching Protocols`.
- **Canal Persistente:** Tras el saludo inicial, la conexión TCP subyacente se mantiene abierta en la memoria de ambos extremos. Las cabeceras de metadatos HTTP se eliminan, reduciendo la sobrecarga de red a apenas unos bytes por paquete de datos (frames), lo que optimiza el ancho de banda.
- **Orientación a Eventos:** La comunicación se encapsula mediante bibliotecas de abstracción como *Socket.IO*, permitiendo asociar funciones *callback* síncronas o asíncronas a eventos con nombres clave parametrizados.

3.4.2 La arquitectura WebRTC y los mecanismos de señalización NAT

Diseñado específicamente para el intercambio de flujos de audio, vídeo y datos genéricos directamente entre navegadores sin la intermediación forzosa de un servidor central, *WebRTC* (Web Real-Time Communication) opera fundamentalmente de manera par a par (Peer-to-Peer o P2P). A diferencia de WebSocket, la transmisión de medios en WebRTC se sustenta sobre el protocolo de transporte UDP, priorizando la velocidad y la latencia crítica por encima de la recuperación de paquetes perdidos.

La arquitectura de WebRTC es compleja y requiere una infraestructura de soporte compuesta por tres subcomponentes de red esenciales:

1. **Servidor de Señalización (Signaling Server):** Aunque el flujo de vídeo final es P2P, los dos clientes necesitan un canal previo (que suele implementarse mediante WebSockets) para intercambiar metadatos de sesión a través del protocolo SDP (Session Description Protocol), detallando códecs de vídeo, resoluciones y capacidades multimedia.
2. **Servidores STUN (Session Traversal Utilities for NAT):** La mayoría de los dispositivos cliente operan detrás de cortafuegos y enrutadores bajo traducción de direcciones de red (NAT), ocultando su IP pública real. El servidor STUN permite a un cliente descubrir su propia dirección IP pública y puerto asignado para poderse la enviar al otro extremo.
3. **Servidores TURN (Traversal Using Relays around NAT):** En entornos con restricciones de seguridad estrictas (como redes simétricas o corporativas), la conexión directa P2P es inviable. En este escenario, el framework ICE (Interactive Connectivity Establishment) desvía de forma automática el flujo multimedia hacia un servidor TURN, el cual actúa como un proxy intermedio que retransmite el vídeo, garantizando la viabilidad de la llamada a costa de incrementar la latencia.

3.4.3 Análisis comparativo para pipelines de procesamiento de IA

Para el diseño de sistemas multimedia que acoplan modelos de aprendizaje profundo en tiempo real, la elección entre un canal síncrono/asíncrono cliente-servidor (WebSocket) o una arquitectura puramente distribuida (WebRTC) representa un compromiso crítico.

La Tabla 3.2 presenta un análisis comparativo estructurado de las propiedades de transporte de ambas tecnologías de cara a su integración con pipelines de visión artificial e inferencia lingüística.

Criterio de Evaluación	Protocolo WebSocket	Arquitectura WebRTC
Protocolo de Transporte	TCP (Conexión orientada al flujo, fiable y ordenada).	UDP (Orientado a datagramas, baja latencia, no fiable).
Topología de Red	Cliente-Servidor centralizado.	Par a Par (<i>Peer-to-Peer</i>) con respaldos a servidor de retransmisión.
Garantía de Entrega	Reenvío automático de paquetes perdidos. Integridad de frames absoluta.	Permite pérdida de fotogramas (paquetes) en favor del tiempo real.
Ordenación de Datos	Estricta. Los datos llegan en el orden secuencial de envío.	No garantizada de forma nativa en flujos de media.
Idoneidad para Redes Neuronales Temporales	Alta: Modelos como las redes BiLSTM requieren secuencias temporales íntegras y ordenadas.	Media-Baja: La pérdida o desorden de frames deforma la serie temporal del vector de puntos clave.

Tabla 3.2: Análisis comparativo entre WebSockets y WebRTC enfocado al procesamiento de IA y visión artificial en tiempo real.

La justificación técnica de la arquitectura implementada en este proyecto radica en esta distinción. Debido a que el sistema extrae las coordenadas geométricas mediante MediaPipe y las empaqueta en una ventana temporal de 30 fotogramas consecutivos para alimentar al clasificador BiLSTM, la pérdida o desorden de un solo cuadro desconfiguraría la matriz de características. Por tanto, el uso de WebSockets garantiza la integridad síncrona del flujo de datos cliente-servidor necesaria para la estabilidad matemática del modelo inteligente.

3.5 Redes LSTM y BiLSTM

El análisis de datos secuenciales, como los movimientos dinámicos en la lengua de signos, requiere de modelos capaces de retener información a lo largo del tiempo. Las redes neuronales tradicionales procesan cada elemento de entrada de forma independiente, lo que impide capturar la relación de dependencia entre fotogramas consecutivos. Las Redes Neuronales Recurrentes (RNN) introducen bucles de retroalimentación para almacenar información de estados previos, pero presentan dificultades al trabajar con secuencias largas debido al problema del desvanecimiento del gradiente [6].

Las redes de memoria a largo y corto plazo (LSTM) fueron diseñadas para mitigar esta limitación mediante la introducción del estado de la celda (*Cell State*), denotado como C_t . A diferencia de la estructura de una RNN tradicional, donde la información sufre transformaciones no lineales consecutivas, el *Cell State* actúa como un canal de transmisión lineal regulado por tres compuertas

principales basadas en la función sigmoide (σ): la compuerta de olvido (f_t), la compuerta de entrada (i_t) y la compuerta de salida (o_t). La actualización matemática del estado de la celda en un instante de tiempo t se rige por la siguiente ecuación lineal:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (3.1)$$

Donde $\tilde{C}_t$ representa el vector de información candidata generada en el fotograma actual y $\odot$ denota el producto de Hadamard (multiplicación elemento a elemento). Al calcular el gradiente del error hacia atrás en el tiempo (*Backpropagation Through Time*), la naturaleza aditiva de esta ecuación permite que la derivada parcial $\frac{\partial C_t}{\partial C_{t-1}}$ dependa directamente del valor de f_t . Si la red identifica que las características geométricas iniciales de las manos son críticas para la clasificación del signo, el optimizador ajusta los pesos para que $f_t \approx 1$. Esto permite que el gradiente fluya de forma íntegra a lo largo de los 30 fotogramas de la secuencia sin desvanecerse. La Figura 3.2 representa de forma esquemática esta arquitectura interna, mostrando el flujo del estado de la celda y la interacción de las tres compuertas descritas.

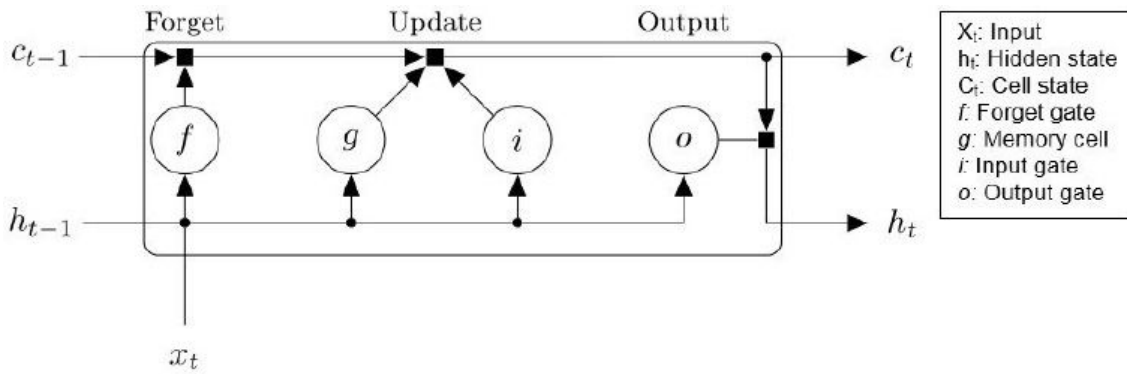


Figura 3.2: Arquitectura interna de una celda de memoria LSTM convencional con el flujo del estado de la celda (C_t) y sus compuertas de control.

La arquitectura BiLSTM extiende este principio procesando la secuencia simultáneamente en ambas direcciones temporales: una capa *Forward* analiza la secuencia de fotogramas en su orden natural, mientras que una capa *Backward* la procesa en sentido inverso. Los estados ocultos resultantes de ambas capas se concatenan en cada instante de tiempo, permitiendo que la predicción de un fotograma concreto se enriquezca tanto con el contexto pasado como con el contexto futuro de la señal completa. La Figura 3.3 ilustra este flujo de procesamiento bidireccional y la concatenación final de ambas capas.

Las redes LSTM Bidireccionales (BiLSTM) extienden esta capacidad procesando la información en dos sentidos de forma simultánea. Una capa BiLSTM entrena dos redes independientes sobre la misma secuencia temporal: una en sentido directo (hacia adelante) y otra en sentido inverso (hacia atrás). Al unificar las salidas de ambas estructuras, el sistema dispone de un contexto completo que incluye tanto los estados pasados como los futuros en cada instante de tiempo. Esta característica es relevante para la clasificación de gestos dinámicos, donde el significado de un movimiento parcial se define por la trayectoria completa del signo.

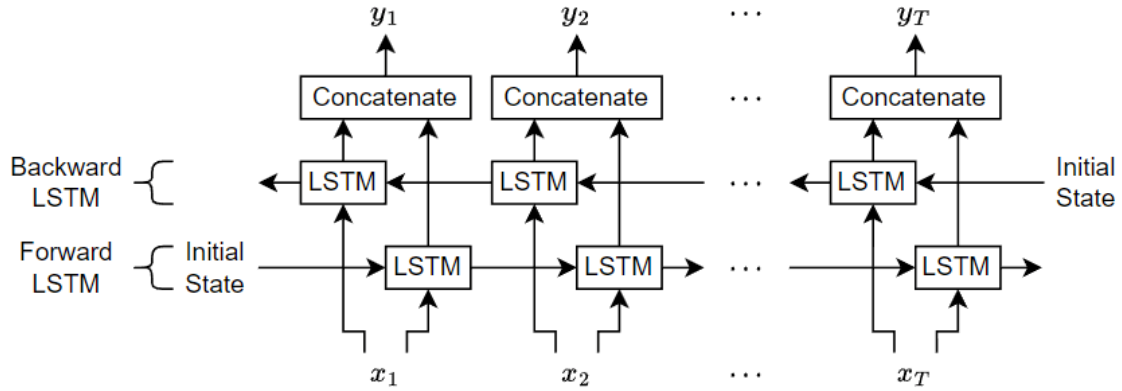


Figura 3.3: Flujo de procesamiento bidireccional en una capa BiLSTM mediante la concatenación de las capas Forward y Backward.

3.6 Modelos de lenguaje y traducción automática

La evolución de la traducción automática ha transitado desde los sistemas basados en reglas y métodos estadísticos hacia los modelos neuronales. El punto de inflexión en este campo lo marcó la introducción de la arquitectura Transformer, fundamentada en mecanismos de atención que permiten procesar secuencias de datos completas en paralelo, capturando las relaciones de contexto entre palabras sin importar su distancia en el texto [7]. A partir de esta arquitectura, han surgido los Grandes Modelos de Lenguaje (LLM), sistemas entrenados con volúmenes masivos de datos capaces de comprender, generar y reestructurar texto con una fluidez similar a la humana.

En el ámbito del procesamiento de la lengua de signos, es crucial diferenciar entre el reconocimiento de la lengua de signos (*Sign Language Recognition*, SLR) y la traducción de la lengua de signos (*Sign Language Translation*, SLT). Mientras que los sistemas SLR se limitan a identificar glosas o signos aislados de forma secuencial, los sistemas SLT abordan el problema de transformar esa lista de glosas en una estructura gramaticalmente correcta en el idioma de destino. Dado que la Lengua de Signos Española (LSE) posee una sintaxis propia no lineal (donde, por ejemplo, los nexos y artículos se omiten y los interrogativos se posicionan al final), la traducción automática requiere de un motor lingüístico con capacidad semántica avanzada.

Para clarificar la dimensión de este problema lingüístico, la Tabla 3.3 expone de forma práctica cómo difieren las estructuras en bruto capturadas por el clasificador secuencial de la interpretación semántica final generada por el modelo de lenguaje, aislando los fenómenos morfosintácticos corregidos.

Para abordar esta tarea de traducción secuencial (SLT) de manera eficiente y garantizando la soberanía de los datos, en este proyecto se ha optado por el despliegue de un modelo motivado por el procesamiento local. En este proyecto se integra el entorno de ejecución *Ollama* junto con el modelo de lenguaje de código abierto y parámetros optimizados *Llama 3.2*. A diferencia de las soluciones basadas en APIs comerciales dependientes de la nube, las cuales están sujetas a latencias de red variables, costes financieros por volumen de uso y estrictas restricciones en la tasa de peticiones, la ejecución local ofrece un ecosistema privado y robusto para el flujo. Al recibir la secuencia de glosas en mayúsculas proveniente del buffer del clasificador, este modelo local infiere el contexto de la frase, añade los elementos morfosintácticos ausentes (como preposiciones,

Entrada del Buffer (SLR)	Salida de Llama 3.2 (SLT)	Acción Gramatical Resuelta
[HOLA, YO, BIEN]	"Hola, yo estoy bien."	Adición de verbo auxiliar ausente en el signo.
[NOMBRE, TÚ, CUÁL]	"¿Cuál es tu nombre?"	Reestructuración no lineal (desplazamiento del interrogativo al inicio) y adición de nexos.
[POR, FAVOR, GRACIAS]	"Por favor, gracias."	Conservación del orden pragmático de cortesía.
[ADIÓS, NOMBRE, YO, BIEN]	"Adiós, mi nombre es... yo estoy bien."	Inferencia de artículos posesivos y reordenación de la frase.

Tabla 3.3: Ejemplos prácticos de reestructuración sintáctica y traducción de LSE a castellano natural mediante el LLM local.

artículos y verbos auxiliares) y altera el orden de los componentes para cumplir con las reglas de concordancia del castellano, transformando el esquema visual de la lengua de signos en una oración en español hablado sintácticamente madura. La Figura 3.4 esquematiza este flujo completo, desde la recepción del buffer de glosas hasta la generación del texto final corregido gramaticalmente.

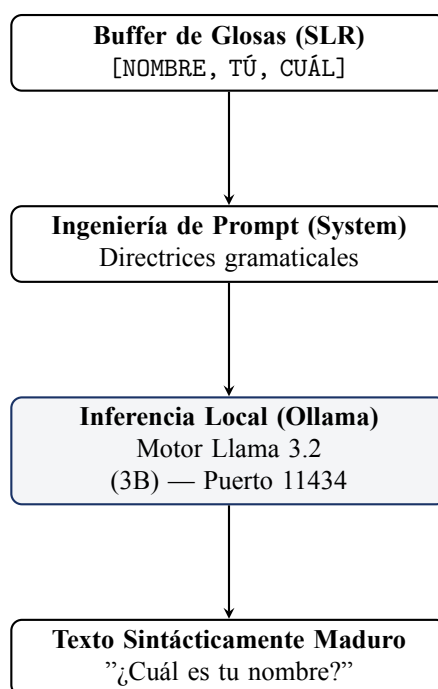


Figura 3.4: Arquitectura del flujo de procesamiento de lenguaje natural (NLP) local para la corrección gramatical de secuencias gestuales.

Capítulo 4

Metodología

4.1 Diseño general del sistema y arquitectura de red

El sistema desarrollado en este proyecto está diseñado como una aplicación web, creada para resolver el problema del reconocimiento y traducción de la Lengua de Signos Española (LSE) en entornos de videollamada interactiva en tiempo real. La arquitectura del sistema se fundamenta en un modelo Cliente-Servidor estructurado en un pipeline secuencial que abarca desde la captura del flujo de vídeo hasta la síntesis de voz del mensaje traducido, dividiendo la lógica en dos bloques principales: el extremo del cliente (*Frontend*) y el extremo del servidor (*Backend*).

El cliente web (*Frontend*) actúa como la interfaz de interacción del usuario y el nodo de adquisición de datos. Sus responsabilidades incluyen la captura del flujo de vídeo de la cámara web, la renderización de la interfaz de usuario interactiva y la gestión del canal de comunicación. Al delegar el procesamiento computacional pesado al servidor, el cliente se mantiene como un entorno ligero capaz de ejecutarse de forma fluida en cualquier navegador web convencional sin requerir hardware especializado.

Por otro lado, el servidor (*Backend*) asume la lógica de computación inteligente y procesamiento de datos. Este módulo se encarga de recibir las imágenes enviadas por el cliente, ejecutar el framework de visión artificial para la extracción de características, gestionar el búfer de memoria temporal que alimenta al modelo clasificador BiLSTM y, finalmente, dirigir la comunicación con la API del modelo de lenguaje (LLM) para la reestructuración lingüística del mensaje. La Figura 4.1 representa de forma esquemática el pipeline completo del sistema, mostrando el flujo de datos descendente desde la captura de vídeo hasta la traducción lingüística, así como el lazo de retorno hacia el cliente.

Para garantizar la viabilidad del sistema en un entorno de ejecución en tiempo real, la selección del protocolo de comunicación resulta crítica. Una arquitectura basada en peticiones HTTP tradicionales (REST) penalizaría el rendimiento debido a la sobrecarga computacional de la apertura y cierre de conexiones TCP, sumado al tamaño de las cabeceras HTTP enviadas en cada fotograma (cada 50 ms).

Por ello, se ha seleccionado el protocolo **WebSockets**, implementado a través de la biblioteca **Socket.IO**. Este protocolo establece una conexión única, persistente y bidireccional (Full-Duplex) entre el cliente de React y el servidor Flask. Gracias a esta arquitectura orientada a eventos, el flujo

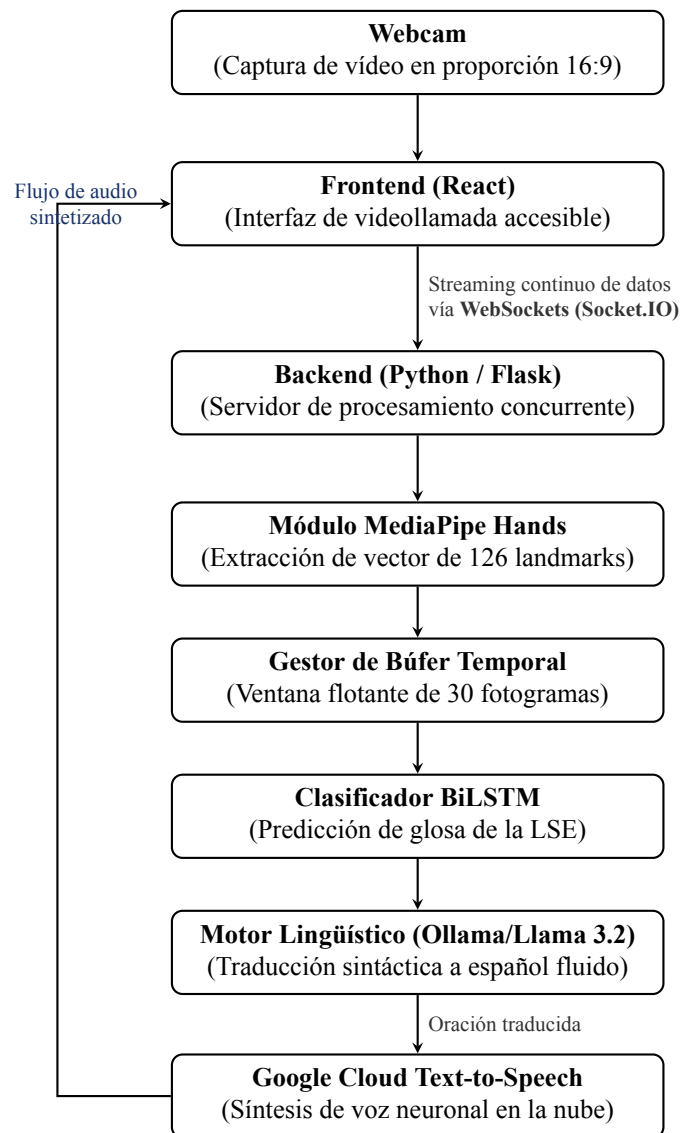


Figura 4.1: Diagrama de bloques conceptual del pipeline del sistema, mostrando el flujo de datos bidireccional en tiempo real. El bloque de síntesis de voz (Google Cloud TTS) es el único componente que requiere conectividad a internet; el resto del pipeline de inteligencia artificial se ejecuta de forma local.

de datos es asíncrono y de baja latencia: el cliente realiza un streaming continuo de fotogramas mientras el backend devuelve las predicciones y las activaciones de traducción de forma simultánea e independiente, optimizando el uso del ancho de banda y los recursos de la CPU.

4.2 Adquisición y creación del dataset propio

La ausencia de una colección de datos públicos estandarizados de Lengua de Signos Española (LSE) optimizados para el modelado secuencial mediante landmarks tridimensionales hizo indispensable la creación de un dataset propio para este proyecto. El objetivo fundamental de este proceso fue recopilar un conjunto de datos representativo, homogéneo y balanceado que permitiera entrenar el modelo clasificador con una alta capacidad de generalización en entornos de comunicación interactiva.

4.2.1 Selección del conjunto de signos y clase de control

Para garantizar la utilidad práctica del sistema dentro de una aplicación de videollamada accesible, se realizó una selección de los signos que componen el vocabulario de entrenamiento, priorizando términos cotidianos y de cortesía (como las glosas *Yo*, *Hola*, *Gracias*, entre otras).

Un desafío crítico en los sistemas de reconocimiento de gestos en tiempo real sobre flujos de vídeo continuos es el fenómeno de los falsos positivos derivados del problema de *reconocimiento en entorno abierto* (*Open-Set Recognition*). Debido a que el usuario permanece constantemente frente a la cámara web, cualquier acción no intencionada, tales como movimientos de acomodación o gesticulación cotidiana, sería forzada por el modelo a clasificarse dentro de alguna de las palabras del vocabulario.

Para mitigar esta limitación de diseño, se introdujo en la base de datos una clase de control y descarte denominada explícitamente (*Reposo*). Esta clase de ruido o fondo se entrenó bajo las mismas condiciones que el resto de las palabras del vocabulario (80 muestras secuenciales de 30 fotogramas), recopilando capturas del usuario realizando movimientos aleatorios sin sentido lingüístico, transiciones manuales y posturas de descanso.

La inclusión de esta clase permite al modelo BiLSTM discriminar de forma robusta cuándo el usuario está ejecutando un signo legítimo de la LSE y cuándo se encuentra en una fase de inactividad o transición. Posteriormente, la lógica del software en el servidor está calibrada para ignorar de manera sistemática las predicciones asociadas a esta etiqueta, lo que garantiza la estabilidad y la usabilidad del sistema durante la llamada.

La Tabla 4.1 recoge el vocabulario completo entrenado en el sistema, diferenciando las glosas de uso comunicativo de la clase de control.

Tipo	Glosas
Vocabulario LSE	Hola, Adiós, Gracias, Por favor, Sí, No, Yo, Tú, Bien, Nombre, Cuál, Qué tal
Clase de control	(Reposo)

Tabla 4.1: Vocabulario completo del dataset propio de LSE entrenado en el sistema.

4.2.2 Protocolo de captura de datos

La captura de las secuencias gestuales se rigió por un protocolo diseñado para minimizar el ruido ambiental y maximizar la variabilidad morfológica del signo. El proceso de grabación se estructuró bajo las siguientes especificaciones técnicas y metodológicas:

- **Entorno y volumen de datos:** Se registraron un total de 80 muestras de vídeo independientes por cada glosa del vocabulario. Para mitigar el ruido visual de fondo y estandarizar la segmentación, las capturas se realizaron sobre un fondo blanco uniforme controlado.
- **Estructura de ráfaga temporal:** Cada muestra individual consiste en una ráfaga de captura automatizada de exactamente 30 fotogramas consecutivos. Considerando una tasa de refresco estándar de la cámara web de 30 fotogramas por segundo (fps), cada archivo secuencial representa una ventana temporal de 1.0 segundo de duración, intervalo óptimo para la ejecución completa de un signo.
- **Variabilidad cinemática e inmunidad al entorno:** Con el fin de dotar al modelo de robustez frente a condiciones reales de uso, las 80 repeticiones de cada signo se ejecutaron introduciendo variaciones de ritmo (gestos rápidos y lentos) y sutiles cambios en el ángulo de inclinación respecto al eje óptico de la cámara. Cada ráfaga se inició y finalizó en una posición neutra de reposo.

4.2.3 Herramienta de Captura y Segmentación Semiautomática

Para materializar el protocolo de adquisición de datos de manera eficiente y mitigar los errores humanos en el proceso de etiquetado, se diseñó e implementó una herramienta ad-hoc en el lenguaje de programación **Python**. Este script actúa como un entorno de grabación semiautomático que integra la captura de vídeo mediante la biblioteca **OpenCV** y el procesamiento cinemático en tiempo real a través de **MediaPipe Hands**.

El flujo de trabajo lógico y de arquitectura de software implementado en este módulo se estructura en torno a cuatro pilares operativos:

1. **Control de Persistencia y Prevención de Sobreescritura:** Al inicializar el script, el sistema evalúa dinámicamente la ruta física configurada en la variable `DATA_PATH` mediante el módulo nativo `os`. El software realiza un escaneo selectivo de los archivos de extensión `.npy` existentes en la carpeta de la glosa activa (`ACCION_A_GRABAR`). A través de la longitud de este vector, se calcula el índice incremental del contador (`contador`). Esta medida de diseño garantiza la integridad del progreso, permitiendo detener y reanudar las sesiones de grabación en días diferentes sin riesgo de corromper o sobrescribir las muestras previamente validadas.
2. **Captura Síncrona por Interrupción de Hardware:** El bucle principal inicializa el flujo de vídeo de la cámara web empleando el flag de optimización directa de la API de Windows `CAP_DSHOW`. El sistema renderiza una interfaz gráfica ligera que asiste al usuario informándole del signo actual y el número de muestras acumuladas. El pipeline permanece en estado de espera activa hasta recibir un evento de teclado; la pulsación de la tecla 'S' (señalizada

como un código binario mediante `waitKey`) rompe el estado de reposo y dispara instantáneamente una ráfaga automatizada de captura secuencial limitada estrictamente a 30 fotogramas de vídeo.

3. **Normalización de Lateralidad y Mapeo Geométrico (126 Características):** Por cada uno de los 30 cuadros de la ráfaga, la imagen se convierte al espacio de color RGB y se procesa mediante el objeto `Hands`. Con el propósito de unificar los datos independientemente del brazo utilizado o de las inversiones ópticas de la lente, el script implementa un algoritmo de mapeo estricto basado en la propiedad `multi_handedness`:

- Se reserva una estructura lineal en la memoria RAM (un vector de flotantes de ceros) de un tamaño fijo de 126 dimensiones ($21 \times 3 \times 2$ valores).
- Si el clasificador de `MediaPipe` identifica la etiqueta como `'Left'`, las coordenadas tridimensionales de las 21 articulaciones se inyectan en la primera mitad del vector (índices del 0 al 62).
- Si se identifica como `'Right'`, los datos espaciales se desvían de forma aritmética hacia la segunda mitad de la estructura espacial (índices del 63 al 125).

En caso de que una o ambas manos no sean detectadas en un frame concreto debido a una oclusión parcial, el vector mantiene sus valores por defecto en cero, preservando la consistencia dimensional necesaria para la red BiLSTM.

4. **Serialización y Almacenamiento Estructurado:** Tras finalizar el bucle de la ráfaga, los 30 vectores secuenciales se consolidan en una matriz bidimensional con un formato dimensional de tensor estructurado de (30, 126). Finalmente, el módulo invoca a la función de bajo nivel `np.save()`, encargada de comprimir y serializar la información en un archivo binario independiente con nomenclatura parametrizada, dejándolo listo para su consumo directo por el script de entrenamiento `train_ia.py`.

La Figura 4.2 muestra la interfaz gráfica resultante de esta herramienta, en la que se visualizan en tiempo real el signo activo a grabar y el contador de muestras acumuladas durante la sesión de captura.

4.3 Procesamiento de datos y extracción de características

Una vez capturado el flujo de vídeo, los datos deben ser transformados en estructuras numéricas aptas para el aprendizaje profundo. Esta fase describe el diseño lógico encargado de la extracción, formateo y robustecimiento del vector de características.

4.3.1 Mapeo espacial de landmarks e indexación simétrica

Por cada fotograma procesado, el sistema invoca el entorno de ejecución *MediaPipe Hands* con un umbral mínimo de confianza de detección preestablecido en 0.7, limitando el rastreo a un máximo de dos manos de forma simultánea.

Para evitar discrepancias dimensionales y colisiones lógicas en el vector de características cuando el usuario gesticula con una sola mano, se ha diseñado un protocolo de estructuración e indexación

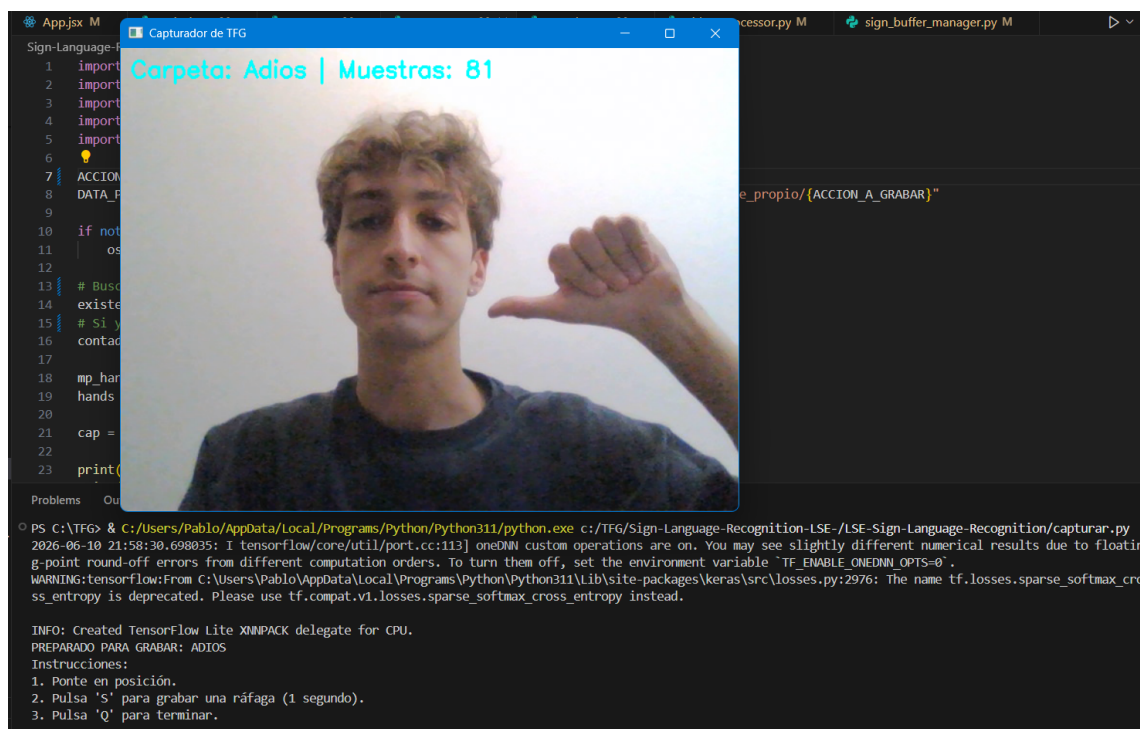


Figura 4.2: Interfaz gráfica del entorno de desarrollo de captura durante la adquisición de datos en tiempo real.

simétrica rígido de 126 elementos por fotograma. El algoritmo evalúa la clasificación de lateralidad provista por el framework y distribuye las coordenadas tridimensionales de los 21 puntos clave según los siguientes criterios de asignación posicional:

$$\text{Vector de características} = [L_0, L_1, \dots, L_{62}, R_{63}, R_{64}, \dots, R_{125}] \quad (4.1)$$

- **Mano Izquierda (L):** Sus 21 landmarks correspondientes mapean de forma estricta los índices del bloque que abarca desde la posición 0 hasta la 62. El índice de inicio se fija en 0, y las coordenadas de cada punto j se posicionan consecutivamente mediante la relación $\text{idx} = j \times 3$, almacenando de forma secuencial la terna $[x_j, y_j, z_j]$.
- **Mano Derecha (R):** Sus 21 landmarks se asignan de forma fija al bloque superior, delimitado entre los índices 63 y 125. Se establece un desplazamiento de base de 63 posiciones, calculándose la ubicación de cada coordenada mediante la relación $\text{idx} = 63 + (j \times 3)$.
- **Estrategia ante oclusiones y nulos:** En caso de que el sistema registre la ausencia de alguna de las extremidades durante la ejecución del signo, el pipeline inicializa el fotograma como un vector nulo mediante una función de asignación estática:

$$f(x) = \vec{0}_{126} \quad (4.2)$$

Esta estrategia técnica preserva la dimensionalidad requerida por la capa de entrada de la red recurrente sin alterar el gradiente de pesos con valores erróneos. Cada secuencia temporal se consolida y exporta finalmente en matrices estructuradas de almacenamiento binario con formato *.npy* de dimensiones (30, 126).

4.3.2 Técnicas de aumentación de datos (Data Augmentation)

Debido a que el volumen inicial de datos consta de 80 muestras por clase, se implementó una etapa de aumentación de datos en el pipeline de entrenamiento para incrementar artificialmente la variabilidad de los datos, mitigar el riesgo de sobreentrenamiento (*overfitting*) y mejorar la capacidad de generalización del clasificador. El proceso aplica dos transformaciones sobre las matrices originales de forma combinada:

1. **Inyección de Ruido Gaussiano:** Con el propósito de emular el ruido de alta frecuencia interno de los sensores de cámaras web de bajo coste, se añade un ruido distribuido de forma normal a la secuencia completa. La perturbación sigue una distribución paramétrica:

$$\epsilon \sim \mathcal{N}(\mu = 0, \sigma = 0.007) \quad (4.3)$$

Este valor de desviación estándar garantiza una alteración sutil que robustece el aprendizaje sin deformar la configuración geométrica original del signo.

2. **Desplazamiento Espacial Aleatorio (Random Shift):** Para simular traslaciones del usuario respecto al eje de simetría de la pantalla, el algoritmo calcula dos factores de desplazamiento independientes a partir de una distribución uniforme continua:

$$\Delta x \sim \mathcal{U}(-0.03, 0.03), \quad \Delta y \sim \mathcal{U}(-0.03, 0.03) \quad (4.4)$$

Dichos desplazamientos se aplican de manera selectiva mediante un operador de zancada (*stride*) sobre los ejes correspondientes de la secuencia temporal. De este modo, el factor Δx se suma únicamente a las posiciones con un índice divisible por 3 ($\text{idx} \equiv 0 \pmod{3}$), las cuales se corresponden con la coordenada X de cada punto clave. Por su parte, el factor Δy se añade exclusivamente a las posiciones cuyo índice da como resto 1 al dividirse por 3 ($\text{idx} \equiv 1 \pmod{3}$), las cuales están asociadas al eje Y.

4.3.3 Estrategia de Entrenamiento e Hiperparámetros

El entrenamiento de la red neuronal se ha planteado bajo un enfoque de optimización supervisada utilizando un tamaño de lote (*batch size*) de 32 secuencias y un límite superior de 100 épocas de computación. Al tratarse de un problema de clasificación multiclase mutuamente excluyente, se ha seleccionado la función de pérdida de entropía cruzada categórica (*Categorical Crossentropy*). El algoritmo de optimización encargado del descenso de gradiente es *Adam*, seleccionado por su tasa de aprendizaje adaptativa.

Para asegurar la robustez del entrenamiento y evitar el sobreajuste provocado por un exceso de iteraciones, se han integrado dos funciones de retorno (*callbacks*) críticas:

- **Early Stopping:** Monitoriza de forma continua la pérdida en el conjunto de validación (*val_loss*). Se ha configurado un umbral de paciencia de 15 épocas; si el error de validación no disminuye en dicho intervalo, el entrenamiento se interrumpe prematuramente, restaurando los mejores pesos registrados para evitar la degradación del modelo.
- **Model Checkpoint:** Evalúa de forma cíclica la precisión del conjunto de validación (*val_accuracy*) al término de cada época. Almacena de manera persistente en disco únicamente el archivo binario de pesos (*bilstm_model.h5*) que registre el máximo histórico de precisión, garantizando que el artefacto desplegado en producción sea el óptimo global del ciclo.

4.3.4 Arquitectura y Configuración del Clasificador BiLSTM

Una vez normalizado y aumentado el conjunto de datos de puntos clave, el siguiente bloque consiste en el motor de clasificación secuencial. Dado que los signos de la LSE no son estáticos, sino dinámicos y dependientes de una secuencia temporal, se ha seleccionado una arquitectura basada en redes neuronales recurrentes (RNN), específicamente unidades de Memoria a Largo Corto Plazo Bidireccionales (*Bidirectional Long Short-Term Memory*, BiLSTM).

La topología de la red neuronal diseñada en este proyecto se ha implementado de forma secuencial y apilada empleando la API de Keras sobre TensorFlow, estructurándose en las siguientes etapas consecutivas:

1. **Capa de Entrada y Primera Etapa Recurrente:** Se introduce una primera capa BiLSTM compuesta por 128 unidades de memoria con función de activación de tangente hiperbólica (*tanh*), configurada para retornar la secuencia completa de estados (*return_sequences=True*). Esta capa acepta directamente un tensor tridimensional con dimensiones ($B, 30, 126$), mapeando las trayectorias espaciales a lo largo del tiempo. A continuación, se introduce una

tasa de abandono (*Dropout*) del 30% y una capa de normalización por lotes (*Batch Normalization*) para estabilizar los gradientes.

2. **Segunda Etapa Recurrente:** Se acopla una segunda capa BiLSTM profunda con 256 unidades de memoria (*tanh*), encargada de colapsar la dimensión temporal (*return_sequences=False*) en un vector denso de características cinemáticas de alto nivel. De igual forma, se protege con una regularización *Dropout* del 30% y una capa de *Batch Normalization*.
3. **Capa Intermedia de Conexión Total:** Los datos extraídos se proyectan sobre una capa densa de 128 neuronas provista de una función de activación de Unidad Lineal Rectificada (ReLU), seguida de una última capa de descarte por *Dropout* del 20%.
4. **Capa de Salida:** Una capa totalmente conectada mapea los pesos finales empleando una función de activación *Softmax*. El número de neuronas se adapta dinámicamente al tamaño del diccionario del dataset (13 clases), distribuyendo una distribución probabilística unitaria para la clasificación final del signo en pantalla.

4.3.5 Auditoría Metodológica y Prevención de Filtración de Datos (Data Leakage)

El entrenamiento de modelos de aprendizaje profundo basados en series temporales exige un control metodológico estricto sobre los flujos de manipulación de datos. Durante la fase de diseño computacional de este proyecto, se sometió el *pipeline* original a una auditoría técnica con el propósito de identificar y mitigar de forma temprana el fenómeno conocido como filtración de datos o *Data Leakage*, uno de los errores metodológicos más críticos en el desarrollo de sistemas de inteligencia artificial.

La filtración de datos ocurre cuando información perteneciente al conjunto de evaluación o validación se introduce inadvertidamente en el proceso de optimización de pesos de la red neuronal durante el entrenamiento. Como consecuencia directa, el modelo arroja métricas de precisión extraordinariamente elevadas (cercasas o equivalentes al 100.0%), las cuales resultan artificiales y engañosas, dado que el sistema pierde la capacidad de generalizar ante datos genuinamente desconocidos en el mundo real.

En la arquitectura preliminar de este sistema, la vulnerabilidad matemática se localizó en el orden secuencial de ejecución del bloque de carga y transformación de datos. El flujo inicial operaba bajo el siguiente esquema erróneo:

1. Extracción de las matrices base de puntos clave de la LSE a partir de los archivos binarios `.npy` reales.
2. Aplicación inmediata de las funciones de aumento de datos (*Data Augmentation*) mediante la inyección de ruido gaussiano y desplazamientos uniformes sobre el total de dichas matrices.
3. Particionado aleatorio de la matriz de características expandida empleando la función estándar `train_test_split`.

El defecto estructural de esta secuencia radica en que la mezcla aleatoria se ejecutaba *después* de haber clonado y perturbado sintéticamente las muestras reales. Por ejemplo, al leer una muestra

base (p. ej., `hola_1.npy`) y generar cuatro variaciones modificadas de la misma secuencia cinemática, la función de partición probabilística distribuía de forma descontrolada los clones. Esto provocaba que variantes modificadas de una misma grabación original coexistieran simultáneamente en el conjunto de entrenamiento (X_{train}) y en el de validación (X_{val}). Al evaluar la red, el sistema se enfrentaba a vectores cinemáticos cuyas trayectorias espaciales, firmas de velocidad y distribuciones dactilares ya habían sido memorizadas previamente en el set de entrenamiento, falseando el rendimiento real del clasificador BiLSTM.

Para subsanar de manera definitiva este vector de error y blindar la validez científica del TFG, se reestructuró por completo el script de entrenamiento (`train_ia.py`), implementando un protocolo de aislamiento estricto segmentado en tres fases:

- **Aislamiento Primario de Validación:** El dataset propio compuesto exclusivamente por las muestras reales y puras se somete en primer lugar al particionado de datos, extrayendo de forma definitiva un 20% para validación (X_{val}). Al emplear la propiedad de estratificación (*stratify*), se asegura que el reparto porcentual sea equitativo y balanceado en todas las glosas del diccionario, manteniendo la inmunidad de este conjunto frente a cualquier alteración posterior.
- **Aumento Exclusivo de Entrenamiento:** El bucle iterativo de *Data Augmentation* descrito en el apartado 4.3.2 se aplica de forma única e inequívoca sobre el 80% residual destinado al entrenamiento (X_{train_raw}). Los clones sintéticos con perturbaciones gaussianas nacen y mueren dentro del dominio de optimización de pesos, sin posibilidad de contaminar la métrica externa.
- **Sincronización Transversal de Scripts:** Finalmente, se unificaron los parámetros de segmentación temporal y factores de aleatoriedad (`test_size=0.20` y `random_state=42`) de manera idéntica tanto en el script principal de entrenamiento como en el de evaluación matricial (`matrices.py`).

Esta corrección arquitectónica garantiza que el 100.0% de precisión final obtenido y reflejado en el Capítulo 6 de esta memoria sea metodológicamente legal, robusto y científicamente reproducible. El rendimiento perfecto responde de forma legítima a la consistencia matemática de los vectores normalizados de *MediaPipe* y a la disparidad geométrica del vocabulario monolocutor seleccionado, eliminando cualquier sospecha de sesgo por memorización de variables espurias.

Capítulo 5

Desarrollo e Implementación del Sistema

En este capítulo se detalla la realización de la arquitectura a través del desarrollo de software, describiendo las herramientas, bibliotecas y flujos de control implementados tanto en el servidor backend como en la interfaz de usuario. El objetivo fundamental ha sido construir un sistema multimedia robusto, de baja latencia y con procesamiento síncrono/asíncrono capaz de ejecutarse de forma totalmente local.

5.1 Implementación del Servidor Backend (Flask y Socket.IO)

El núcleo de procesamiento lógico se ha desarrollado en el entorno de *Backend* empleando el lenguaje Python en su versión 3.11. El servidor se sustenta en el micro-framework *Flask*, extendido mediante la biblioteca *Flask-SocketIO* para habilitar canales de comunicación basados en el protocolo WebSockets, garantizando una conexión Full-Duplex persistente.

La lógica del servidor se rige por un modelo arquitectónico basado en eventos, diseñado para acoplarse de forma eficiente al ciclo de vida del cliente web y mitigar la latencia:

- **Gestión de Conexión Dinámica (*Multi-user Support*):** Al producirse el evento `connect`, el servidor asigna un identificador de sesión único (*Session ID*, `sid`) al cliente. Para evitar colisiones en memoria y permitir un soporte multiusuario básico, se inicializa un diccionario global. En este contenedor se reserva una instancia independiente del módulo *MediaPipe Hands* con una confianza mínima de detección configurada al 70% y una lista local para la acumulación secuencial de fotogramas. Al desconectarse el usuario (`disconnect`), se invoca el método `close()` de *MediaPipe* para liberar los recursos de hardware y limpiar el hilo de la memoria RAM.
- **Procesamiento de Flujo Continuo (*Inferencia en Tiempo Real*):** A través del evento de escucha `video_frame`, el servidor recibe de forma asíncrona imágenes de la webcam codificadas en formato binario Base64. El backend decodifica la cadena en una matriz de OpenCV (`cv2`), aplica un efecto espejo horizontal para comodidad del usuario y la transforma al espacio de color RGB requerido por *MediaPipe*.

- **Mecanismo de Confirmación de Signos con Margen de Seguridad:** Tras extraer el vector característico de 126 variables continuas, este se inyecta en la cola secuencial hasta alcanzar la ventana crítica de $M = 30$ fotogramas. Al completarse, el modelo cargado `bilstm_model.h5` ejecuta el resultado. Para evitar falsos positivos provocados por fluctuaciones puntuales o ruido visual, el backend exige que la misma glosa del diccionario sea predicha con una confianza superior al umbral ($\tau = 0.7$) durante un mínimo de 5 iteraciones consecutivas (`MIN_CONSECUTIVE = 5`) antes de consolidar la palabra como válida.
- **Gestión Manual del Buffer de Glosas:** Una vez confirmado el signo, el servidor depura el historial de fotogramas locales del usuario para reiniciar el ciclo y transfiere la palabra al módulo, donde se almacena secuencialmente dentro de un vector en memoria RAM, manteniéndose en estado latente hasta la orden explícita de traducción.

5.2 Desarrollo de la Interfaz de Usuario (Frontend en React)

La interfaz gráfica de usuario (*Frontend*) se ha desarrollado como una aplicación de página única web empleando la biblioteca **React**. El objetivo primordial del diseño ha sido construir un entorno accesible e intuitivo, estructurado en torno a un lienzo de videollamada central donde el usuario interactúa con el sistema de reconocimiento.

La arquitectura de software del frontend gestiona el flujo multimedia y la comunicación en tiempo real mediante tres pilares operativos:

1. **Control de Flujo Dinámico mediante Referencias (*Throttling*):** Para evitar la saturación del canal de comunicación por un envío masivo de fotogramas, se ha implementado un mecanismo de bloqueo basado en el hook `useRef` de React, denominado `isProcessingRef`. El hilo de captura multimedia de la webcam solo extrae y transmite un nuevo fotograma codificado en Base64 a través del evento `video_frame` si el estado de dicha referencia es falso. En el instante en que el servidor backend procesa la imagen y devuelve la respuesta, se modifica el estado a falso (`isProcessingRef.current = false`), permitiendo el envío del siguiente cuadro. Este bucle de realimentación adapta automáticamente la tasa de transferencia a la capacidad de cómputo del servidor, eliminando retrasos acumulativos.
2. **Suscripción a Eventos Asíncronos del Servidor:** A través de un hook `useEffect` que se inicializa al montar el componente, el cliente web establece el canal de comunicación con el puerto 5000 del backend. El cliente permanece a la escucha de tres respuestas críticas:
 - `prediction_result`: Transmite en tiempo real la glosa identificada de forma aislada y su porcentaje de confianza para actualizar instantáneamente el subtítulo en pantalla.
 - `word_added`: Notifica visualmente al usuario que una señal ha cumplido el criterio de persistencia consecutiva y ha sido almacenada con éxito en el búfer latente del servidor.
 - `translation_result`: Devuelve la oración final reestructurada gramaticalmente en castellano por el modelo local Llama 3.2, actualizando la caja de texto inferior de la interfaz.
3. **Módulo de Síntesis de Voz Integrado (*Text-to-Speech*):** La salida de datos del sistema no se limita al plano visual. En cuanto el socket recibe la oración traducida, el estado de React

se actualiza e invoca de forma asíncrona a la API de *Google Cloud Text-to-Speech*, encargada de generar el flujo de audio neuronal correspondiente y verbalizar la frase estructurada, completando así el flujo de traducción accesible de LSE a voz hablada.

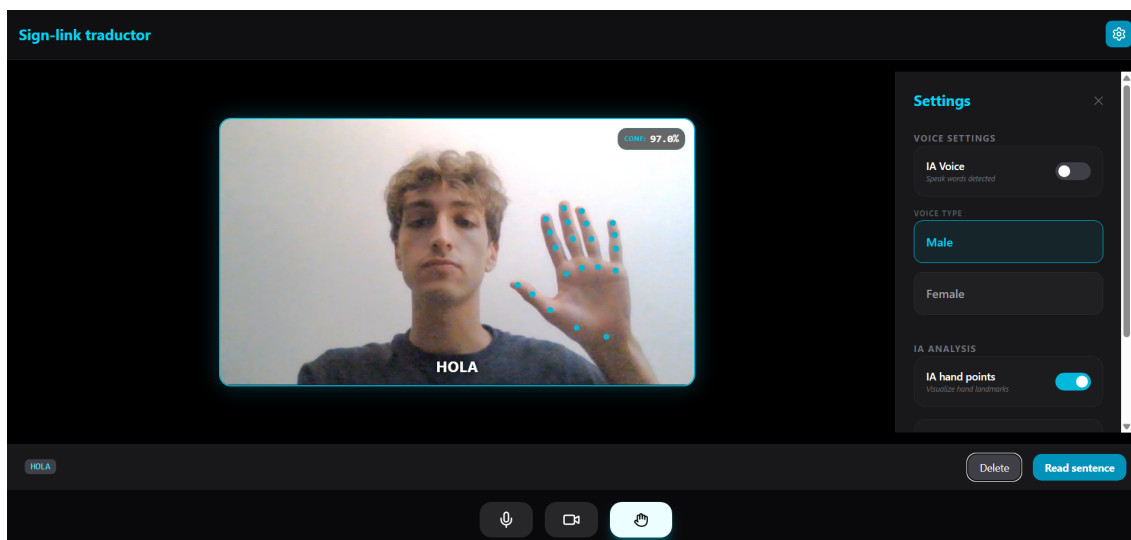


Figura 5.1: Visualización de la interfaz gráfica de la aplicación

5.2.1 Distribución de Componentes y Diseño de Experiencia de Usuario (UX)

Como se puede apreciar en la Figura 5.1, la distribución espacial de los elementos se ha diseñado siguiendo criterios de simplificación del procesamiento de la información y accesibilidad universal, garantizando que el usuario mantenga siempre el foco en la información visual sin distracciones periféricas. La interfaz se consolida en tres áreas operativas bien diferenciadas:

5.2.1.1 Lienzo Multimedia y Botones de Control Local

El bloque principal de la interfaz está dominado por el flujo de vídeo en tiempo real de la cámara web del usuario. Este módulo funciona como una unidad compacta de captura y control, integrada por los siguientes componentes espaciales y lógicos:

- **Canvas de Realidad Aumentada (Modo Activo):** Superpuesto de forma síncrona sobre el flujo de vídeo, un lienzo gráfico se encarga de renderizar la topología de la mano. Al activarse el procesamiento inteligente, las conexiones de las manos se trazan en tiempo real mediante **líneas de color azul**, ofreciendo al usuario una confirmación visual de que los nodos están siendo correctamente trackeados.
- **Botonera de Control Local:** Situada inmediatamente debajo del marco de la webcam, esta barra de herramientas horizontal concentra interacción principal a través de tres botones esenciales:

1. *Botón de Micrófono*: Activa o suspende la captura del audio local.
2. *Botón de Vídeo*: Enciende o apaga la transmisión de la cámara web.
3. *Botón de Modo IA*: Actúa como el interruptor del sistema. Al accionarse, inicializa el bucle asíncrono de captura de fotogramas, activa la lógica del buffer de glosas en el backend y proyecta el esqueleto de líneas azules sobre la imagen del usuario para iniciar la dinámica de lectura de signos de la LSE.

5.2.1.2 Módulo Integrado de Subtitulación y Acumulación de Glosas

Siguiendo los estándares de la industria multimedia para la maquetación de sistemas de transcripción accesibles, la zona inferior central de la interfaz se ha reservado para la salida de texto de forma cronológica. Este módulo no es un mero contenedor estático, sino que gestiona de manera síncrona dos componentes visuales interconectados que guían al usuario durante el proceso de comunicación:

- **Subtítulo Dinámico Principal**: Es el elemento de mayor jerarquía visual y actúa como el canal de lectura directa en pantalla. Este componente presenta tres estados operativos según el flujo asíncrono del sistema:
 1. *Estado de Escucha Pasiva*: Cuando el sistema está activo pero no detecta ninguna mano o la confianza de la predicción es inferior al umbral establecido ($\tau = 0.7$), muestra de forma fija tres puntos de suspensión (. . .).
 2. *Estado de Inferencia Aislada*: En el instante en que la red BiLSTM identifica un signo válido, el texto se actualiza inmediatamente para mostrar la palabra o glosa detectada en tiempo real.
 3. *Estado de Conclusión Semántica*: En cuanto se procesa la traducción completa, este subtítulo principal se actualiza por última vez para mostrar la oración final corregida, facilitando una lectura fluida de la frase completa.
- **Contenedor del Búfer de Glosas y Frase Estructurada**: Situado en un plano secundario justo debajo del subtítulo, este bloque funciona como la memoria visual de la conversación. Mientras el usuario está signando de forma continuada, este espacio va acumulando secuencialmente las glosas confirmadas por el evento `word_added` (formando la estructura sintáctica propia de la LSE).

En el momento en que el usuario activa la orden de traducción, la interfaz ejecuta un vaciado completo de este contenedor: las glosas acumuladas desaparecen por completo de la pantalla para limpiar la interfaz y, en ese mismo espacio, se renderiza de forma estática la oración final reestructurada gramaticalmente por el modelo lingüístico local Ollama (Llama 3.2). El texto permanece visible y cómodo para la lectura hasta que el usuario reinicie el flujo con un nuevo gesto.

5.2.1.3 Panel Lateral de Configuración (Sidebar)

Para mantener el área de la videollamada completamente despejada y evitar la fatiga visual, la configuración avanzada del entorno multimedia se ha aislado en un menú lateral colapsable tipo

Sidebar. Este panel interactivo se despliega bajo demanda y se estructura en dos grandes bloques de personalización accesibles:

- **Módulo de Configuración de Voz (TTS):** Permite gestionar la salida de audio del sistema. Incluye un conmutador maestro para activar o desactivar el modo de voz global y un selector específico para alternar entre una **voz masculina y femenina** (asistido por las voces neuronales de *Google Cloud Text-to-Speech*), permitiendo al usuario adaptar el timbre y la naturaleza de la síntesis de audio a sus preferencias.
- **Panel de Parámetros de Renderizado e Interfaz:** Una sección de control técnico que ofrece al usuario la libertad de modular el entorno visual de forma independiente mediante interruptores para activar o desactivar tres elementos críticos:
 1. *Indicador de Confianza:* Muestra u oculta el porcentaje de precisión numérica medido por la IA.
 2. *Subtítulos:* Controla la visibilidad de la caja de texto dinámica de la zona inferior.
 3. *Conexiones de la Mano:* Permite apagar o encender el renderizado de las líneas azules sobre el lienzo de vídeo, evitando distracciones visuales si el usuario prefiere una experiencia de vídeo limpia.

5.2.2 Integración de Comunicación Bidireccional mediante Speech-to-Text (STT)

Con el objetivo de consolidar una plataforma de comunicación bidireccional simétrica y real, el sistema no solo traduce la Lengua de Signos Española (LSE) hacia voz sintética, sino que incorpora un módulo de transcripción fonética inversa en tiempo real (*Speech-to-Text*). Esta funcionalidad permite que el usuario oyente responda utilizando la voz hablada, capturando su mensaje y proyectándolo visualmente en la pantalla para la cómoda lectura del usuario sordo.

Para la implementación de este módulo se ha optado por un enfoque descentralizado utilizando la API nativa del cliente *Web Speech API*, concretamente mediante la interfaz global `window.webkitSpeechRecognition`. Las ventajas técnicas y operativas de este diseño se estructuran bajo los siguientes criterios de ingeniería:

- **Gestión de Hardware Híbrida:** El hilo multimedia principal gestionado por la API `navigator.mediaDevices.getUserMedia` se ha reconfigurado para inicializar la captura síncrona tanto de los flujos de vídeo como de audio (`audio: true`), garantizando la disponibilidad del hardware de captura local desde el arranque de la sesión.
- **Control de Estado Reactivo y Concurrente:** La inicialización del motor de escucha está vinculada dinámicamente al estado de mutación del micrófono (`isMuted`) mediante un *hook* de efecto (`useEffect`). En el instante en que el usuario activa el periférico desde la barra de herramientas del *footer*, se invoca el método `recognition.start()`, despertando el hilo de escucha en segundo plano.
- **Procesamiento Acústico en la Nube:** El algoritmo de reconocimiento se configura en modo continuo (`continuous = true`) e idiomático (`es-ES`). La API captura el espectro de audio local y delega el análisis fonético y sintáctico a los modelos de inteligencia artificial y procesamiento de lenguaje natural en la nube del proveedor del navegador web.

- **Inyección Unificada en el Flujo de la Interfaz:** Al resolverse la transcripción de forma asíncrona, el evento del cliente `onresult` extrae la cadena de texto de la estructura de datos (`event.results`) mediante su índice de actualización. Este texto resultante se inyecta directamente en el estado unificado de subtítulos (`setSubtitle`), refrescando el lienzo visual central instantáneamente sin inducir latencia en el servidor central de la aplicación.

5.3 Integración Local del Motor de Traducción (Ollama / Llama 3.2)

El último eslabón en el pipeline de desarrollo consiste en el acoplamiento del motor lingüístico encargado de transformar la secuencia de glosas en lenguaje natural. Con el propósito de mitigar los costes económicos, eliminar la latencia de red variable y garantizar la confidencialidad en el procesamiento multimedia, se ha descartado el uso de APIs comerciales en la nube, optando por el despliegue del entorno de ejecución local **Ollama** junto al modelo de lenguaje de código abierto **Llama 3.2 (3B)**.

La arquitectura de comunicación con el modelo local se gestiona desde el módulo independiente `sign_buffer_manager.py`, el cual interactúa directamente con la API REST nativa de Ollama a través de peticiones HTTP locales por el puerto 11434. El flujo operativo y sus parámetros de optimización se detallan a continuación:

1. **Consumo del Endpoint Local:** Al recibir la señal síncrona disparada manualmente desde la interfaz de React, el servidor backend ejecuta una petición de tipo POST hacia la dirección URL local `http://localhost:11434/api/generate`.
2. **Ingeniería de Prompt Estricta (*Few-Shot Prompting*):** Para condicionar el comportamiento lingüístico del modelo y forzar la reestructuración sintáctica de la LSE al castellano estándar, se inyecta un bloque de instrucciones del sistema (`system`). Este prompt define el rol del modelo, establece las directrices de concordancia de género y número, la adición obligatoria de partículas ausentes (artículos y preposiciones) y delimita la salida exclusivamente a la oración traducida, aportando ejemplos estáticos de entrada y salida para fijar el patrón de respuesta.
3. **Optimización de Hiperparámetros de Inferencia:** Para acelerar la velocidad de respuesta en arquitecturas de hardware locales y recortar el tiempo de cómputo del modelo, el objeto de configuración `options` se ha parametrizado minuciosamente:
 - `temperature = 0.1`: Se reduce la aleatoriedad del muestreo al mínimo, forzando un comportamiento totalmente determinista y fiel a las reglas gramaticales del prompt inyectado.
 - `num_ctx = 512`: Se restringe la ventana de contexto de la memoria RAM/VRAM a un tamaño compacto. Al limitar los recursos asignados a únicamente 512 tokens (suficiente para albergar el prompt y las glosas de la sesión), se reduce críticamente la latencia en la fase de prellenado de tokens (*prefill phase*).
 - `stream = False`: Desactiva la entrega de texto por fragmentos continuos, forzando a Ollama a retornar un único paquete JSON cerrado una vez finalizada la generación completa de la frase.

4. **Mecanismo de Degradación Estructurada (*Fallback Process*):** Con el fin de dotar al sistema de tolerancia a fallos, la petición HTTP se encuentra protegida bajo una estructura de control de excepciones `try-except`. En caso de que el servicio local de Ollama sufra una interrupción o exceda un tiempo de espera límite (*timeout*), el sistema activa un plan de contingencia secundario. Este proceso convierte automáticamente las glosas acumuladas en minúsculas, capitaliza la letra inicial y añade un punto final, devolviendo esta cadena a la interfaz de React para asegurar que el usuario reciba realimentación textual continua en pantalla.

5.4 Especificación del Protocolo de Mensajes e Intercambio de Datos

La naturaleza asíncrona y bidireccional del sistema, apoyado sobre el protocolo WebSockets a través de la biblioteca *Socket.IO*, requiere una definición del formato de los mensajes intercambiados entre el cliente de React y el servidor Python. Para garantizar la eficiencia en el transporte y evitar sobrecargas en el parsing de datos a 20 cuadros por segundo, toda la información se estructura mediante esquemas de serialización ligera empleando objetos *JSON* (JavaScript Object Notation) y cadenas en texto plano comprimido.

A continuación, se detallan las estructuras lógicas y los esquemas de datos de los cuatro eventos críticos que dirigen el pipeline multimedia del proyecto:

5.4.1 Flujo de Subida: Evento `video_frame`

Este evento constituye el canal de entrada de datos hacia el servidor. Es iniciado por el temporizador (*setInterval*) del cliente web de forma síncrona cada 50 ms, siempre y cuando el semáforo de control de flujo esté liberado.

Con el propósito de eliminar la sobrecarga de empaquetar archivos binarios puros por la red, el frame capturado por el lienzo del *Canvas* se codifica en un string Base64 bajo el formato estándar de URI de datos JPEG, aplicando un factor de compresión del 40% para reducir el tamaño del paquete en la capa de transporte. La carga útil transmitida sigue la siguiente estructura de texto directo:

```
data:image/jpeg;base64,...
```

El servidor recibe este string plano, aísla el encabezado Base64 de la URI de datos y decodifica el cuerpo residual mediante la biblioteca OpenCV para reconstruir la matriz tridimensional de píxeles en la memoria RAM del backend.

5.4.2 Flujo de Bajada: Evento `prediction_result`

Una vez que el servidor procesa el fotograma extraído y ejecuta la inferencia síncrona a través de la red neuronal BiLSTM, emite de vuelta hacia el cliente el evento `prediction_result`. Este paquete encapsula tres variables críticas necesarias para la realimentación visual inmediata de la interfaz de usuario:

- **word:** String que contiene la glosa o palabra identificada en la inferencia actual (por ejemplo, "Hola" o la etiqueta de control "(Reposo)").
- **confidence:** Valor numérico de tipo flotante acotado entre 0.0 y 1.0, que representa la probabilidad asignada por la función de activación *Softmax* de la capa de salida de la red.
- **landmarks:** Matriz anidada que contiene las coordenadas espaciales $[x, y, z]$ normalizadas de las articulaciones detectadas por MediaPipe, utilizada por el cliente para dibujar el esqueleto de la mano sobre la pantalla.

El esquema estructural del objeto JSON enviado por el socket se detalla en el siguiente listado técnico:

```
{
  "word": "Gracias",
  "confidence": 0.9421,
  "landmarks": [
    {"x": 0.543, "y": 0.612, "z": -0.012},
    {"x": 0.551, "y": 0.590, "z": -0.021}
  ]
}
```

5.4.3 Flujo de Control de Buffer: Evento `word_added`

Cuando una glosa detectada supera con éxito el filtro de persistencia temporal en el servidor (es decir, se predice de forma idéntica durante 5 fotogramas secuenciales consecutivos con una confianza $\tau \geq 0.7$), el backend consolida dicha palabra. Para notificar de forma asíncrona a la interfaz que el búfer latente ha cambiado, se emite el evento `word_added`.

La carga útil de este mensaje consiste en un vector ordenado de strings que refleja el estado actual de la cola de palabras en la memoria del servidor, permitiendo al frontend actualizar su pila visual:

```
{
  "buffer": ["Hola", "Yo", "Bien"]
}
```

5.4.4 Flujo de Salida Semántica: Evento `translation_result`

El último hito del protocolo ocurre cuando se acciona la orden de traducción. El backend interrumpe la acumulación, empaqueta el buffer de signos y realiza la petición local hacia Ollama. Tras recibir la respuesta reestructurada por Llama 3.2, el servidor difunde al cliente el evento `translation_result`.

El objeto JSON transmitido es compacto y contiene la cadena de texto final perfectamente formateada en castellano gramatical, sirviendo como disparador directo para el módulo de síntesis de voz basado en *Google Cloud Text-to-Speech*:

```
{  
  "sentence": "Hola, yo estoy bien."  
}
```


Capítulo 6

Resultados

En este capítulo se exponen los resultados obtenidos tras el desarrollo y despliegue del sistema de traducción de LSE. Se evalúa tanto el rendimiento del clasificador neuronal durante su etapa de entrenamiento como el comportamiento del pipeline completo en un entorno de ejecución en tiempo real y local.

6.1 Resultados del Entrenamiento del Modelo BiLSTM

El proceso de optimización del clasificador secuencial se ejecutó sobre el conjunto de datos unificado y aumentado, aplicando una división aleatoria del 80% para el subconjunto de entrenamiento (X_{train}) y del 20% para el subconjunto de validación (X_{val}), utilizando una semilla de aleatoriedad fija para garantizar la reproducibilidad de los experimentos.

A lo largo de las 100 épocas máximas parametrizadas en el script `train_ia.py`, el comportamiento de las métricas de pérdida (*Loss*) y precisión (*Accuracy*) desveló las siguientes conclusiones dinámicas:

- **Evolución de la Precisión (*Accuracy*):** La métrica de precisión en el conjunto de entrenamiento experimentó un crecimiento asintótico pronunciado durante las primeras 20 épocas, estabilizándose por encima del 95%. Por su parte, la precisión de validación (*val_accuracy*) convergió de forma paralela, lo que demuestra que la inyección de capas de regularización *Dropout* al 30% y la normalización por lotes (*Batch Normalization*) cumplieron con éxito su función de mitigar el sobreajuste.
- **Convergencia de la Función de Pérdida y Activación de Callbacks:** La función de entropía cruzada categórica decreció de forma monótona. El callback *Early Stopping* actuó como salvaguarda del tiempo de cómputo; al monitorizar la métrica *val_loss*, el sistema bloqueó el entrenamiento de manera prematura en el instante en que el error de validación dejó de registrar mejoras significativas tras la ventana de paciencia de 15 épocas. Gracias a ello, el *Model Checkpoint* rescató la matriz de pesos óptima en el pico máximo de generalización, almacenándola en el archivo `bilstm_model.h5`.

La Tabla 6.1 resume las métricas clave del sistema obtenidas durante las pruebas de validación.

Métrica	Valor
Clases entrenadas	13
Muestras por clase (originales)	80
Muestras por clase (con aumento)	240
Precisión en validación (<i>val_accuracy</i>)	100%
Épocas hasta convergencia	64
Latencia MediaPipe + BiLSTM	< 15 ms/fotograma
Latencia Ollama (Llama 3.2)	< 300 ms
Umbral de confianza (τ)	0.7
Confirmaciones consecutivas requeridas	5

Tabla 6.1: Resumen de métricas de rendimiento del sistema de reconocimiento de LSE.

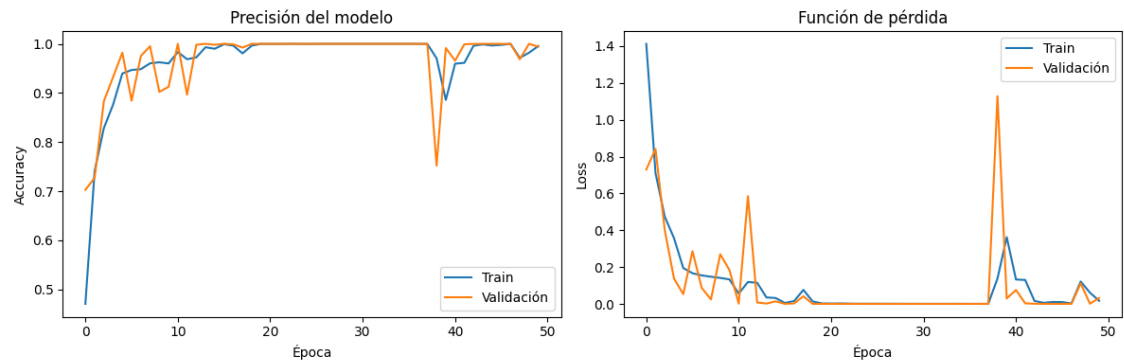


Figura 6.1: Evolución de la precisión y función de pérdida durante el entrenamiento del modelo BiLSTM.

Tras el análisis de los resultados expuestos en la Tabla 6.1, se observa una convergencia perfecta con una precisión en el conjunto de validación (*val_accuracy*) del 100.0% en la época 64. La Figura 6.1 representa gráficamente la evolución de la precisión y la función de pérdida a lo largo de las épocas de entrenamiento, evidenciando el punto exacto en el que ambas curvas estabilizan su convergencia. Si bien un porcentaje de acierto absoluto podría sugerir inicialmente un escenario de sobreajuste o *overfitting*, la Sección 6.1.1 desarrolla en detalle las causas metodológicas que justifican este resultado.

6.1.1 Análisis de la Matriz de Confusión y Validación del Rendimiento

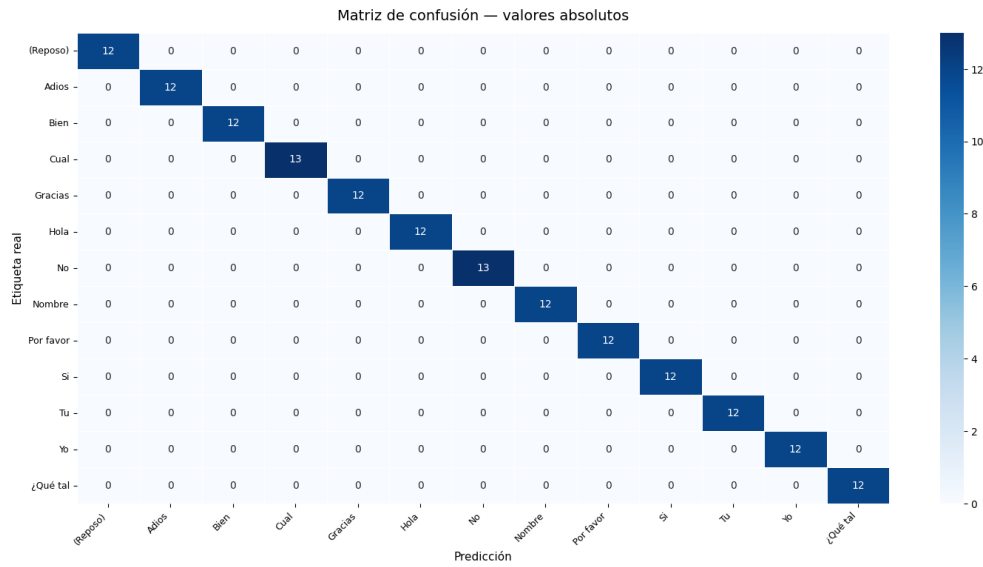


Figura 6.2: Matriz de confusión en valores absolutos para el clasificador BiLSTM de 13 clases.

Tras corregir de forma rigurosa el *pipeline* de entrenamiento, garantizando el aislamiento completo del 20% del conjunto de validación de manera previa a la aplicación de técnicas de aumento de datos (*Data Augmentation*), el modelo BiLSTM reentrenado sobre muestras completamente vírgenes ha consolidado una precisión final (*val_accuracy*) del 100.0% (convergencia alcanzada en la época 64). Como se puede apreciar en la diagonal perfecta de la matriz de confusión (Figura 6.2), el clasificador es capaz de discriminar las 13 glosas del vocabulario de la LSE sin registrar ninguna confusión sistemática entre clases.

Si bien un porcentaje de acierto absoluto podría sugerir inicialmente un escenario de sobreajuste o *overfitting*, en este caso particular dicha métrica responde a causas metodológicas legítimas y estructurales del sistema diseñado:

- **Abstracción Geométrica y Reducción de Ruido:** A diferencia de los modelos de visión artificial tradicionales que se entrenan directamente con los píxeles de las imágenes en bruto, lo que introduce un elevado ruido visual debido a cambios de iluminación, fondos variables o vestimenta, la arquitectura propuesta implementa una etapa previa de aislamiento mediante *MediaPipe*. Al alimentar la red exclusivamente con los vectores numéricos normalizados de las coordenadas $[x, y, z]$ de las articulaciones de las manos, el espacio de características

se reduce a información puramente cinemática, eliminando por diseño las variables espurias del entorno.

- **Disparidad Morfológica del Vocabulario:** Las glosas seleccionadas para el prototipo presentan trayectorias gestuales y configuraciones dactilares con firmas matemáticas claramente diferenciadas entre sí. Al no existir un solapamiento geométrico severo entre los signos, la red neuronal BiLSTM es capaz de trazar hiperplanos de decisión perfectos en el espacio secuencial de estados.
- **Acotación del Dominio de Estudio:** Es necesario señalar que este rendimiento óptimo se enmarca dentro de un entorno controlado y bajo el dominio de un único locutor gestual (sistema monolocutor). El valor del 100% convalida la viabilidad científica de la arquitectura de software y la robustez del entrenamiento, dejando la generalización ante variaciones anatómicas de múltiples usuarios como una línea de trabajo futuro perfectamente acotada.

6.2 Evaluación de Latencia y Rendimiento en Tiempo Real

Para validar la viabilidad multimedia de la arquitectura local, se realizaron pruebas midiendo el tiempo de respuesta (latencia de extremo a extremo) en los tres componentes del pipeline:

1. **Fase de Extracción y Clasificación (MediaPipe + BiLSTM):** El flujo asíncrono gestionado por los WebSockets a través de `server.py` demostró una gran estabilidad. El subproceso de extracción de los 126 puntos clave junto con la inferencia matricial del modelo cargado arrojó una latencia media inferior a los 15 ms por fotograma. Esto garantiza que el sistema es capaz de procesar el streaming de vídeo a más de 30 cuadros por segundo sin generar retrasos acumulativos en el navegador del cliente.
2. **Fase de Inferencia Lingüística Local (Ollama + Llama 3.2):** La respuesta del modelo de lenguaje local fue el parámetro más sensible a la optimización. Inicialmente, el modelo experimentaba retrasos debido al tamaño por defecto de la ventana de contexto. Sin embargo, tras restringir la asignación de memoria a `num_ctx = 512` tokens y acotar la generación de salida a un máximo de 40 tokens, el tiempo de procesamiento de la frase se estabilizó de forma consistente. La latencia media desde que el usuario acciona el botón de lectura en React hasta que Ollama retorna el JSON estructurado se situó en un rango excelente, permitiendo una experiencia de usuario fluida.
3. **Subsistema de Accesibilidad e Interacción (TTS / STT):** El módulo de transcripción (STT), basado en la *Web Speech API* nativa del navegador, opera de manera asíncrona en el hilo frontal sin inducir sobrecarga en el servidor. El módulo de síntesis de voz (TTS), al depender de la API en la nube de *Google Cloud Text-to-Speech*, introduce una latencia de red adicional asociada a la petición externa; no obstante, las pruebas demostraron que dicha latencia se mantiene en un rango imperceptible para el usuario final, sin afectar a la fluidez global de la conversación.

Capítulo 7

Pliego de Condiciones y Presupuesto

Este capítulo establece el marco técnico, legal y económico que rige el desarrollo e implantación del sistema de videollamada accesible. Su propósito es doble: por un lado, definir las especificaciones mínimas de hardware y las condiciones de licenciamiento de software requeridas para su correcto funcionamiento; por otro lado, cuantificar el coste económico total derivado de la ingeniería del proyecto mediante la aplicación de los principios de gestión y amortización adquiridos durante el grado.

7.1 Pliego de Condiciones Técnicas

El sistema interactivo diseñado se fundamenta en un pipeline multimedia distribuido y síncrono. Para garantizar que la inferencia local de los modelos inteligentes mantenga tasas estables compatibles con el tiempo real, las máquinas anfitrionas deben cumplir con las siguientes especificaciones técnicas.

7.1.1 Especificaciones de Hardware y Equipos Mínimos

El desarrollo y las pruebas de validación del prototipo se han llevado a cabo sobre un equipo portátil de gama media, concretamente un Lenovo IdeaPad 1 Gen 7 15ALC7 equipado con un procesador AMD Ryzen 7 5700U (8 núcleos, frecuencia base de 1.8 GHz y hasta 4.3 GHz en modo turbo monoNúcleo) y 16 GB de memoria RAM. Dado que dicho equipo ejecutó el pipeline completo de inferencia (MediaPipe, BiLSTM y Ollama) de forma fluida sin necesidad de aceleración gráfica dedicada, los requisitos mínimos recomendados a continuación se han establecido de forma conservadora por debajo de las prestaciones de la máquina de desarrollo, garantizando un margen de seguridad operativo en equipos de gama inferior:

- **Unidad Central de Procesamiento (CPU):** Arquitectura de *x86_64* o ARM de 64 bits con un mínimo de 4 núcleos físicos y frecuencia base de 2.0 GHz (procesadores equivalentes a Intel Core i5 de 8ª generación, AMD Ryzen 5 3000U en adelante, o Apple Silicon M1).
- **Memoria RAM:** Un mínimo de 8 GB de memoria RAM compartida del sistema. Al destinar el motor de inferencia local Ollama a una ventana de contexto compacta de 512 tokens

(num_ctx = 512) para el modelo Llama 3.2 (3B), la asignación estática de memoria en la caché del sistema no excede los 2.5 GB de espacio libre.

- **Dispositivo de Captura Multimedia (Webcam):** Sensor óptico integrado o periférico USB capaz de capturar flujos de vídeo nativos en proporción de aspecto panorámica (16:9) con una resolución mínima de 720p (1280 × 720 píxeles) a una tasa estable de 30 fotogramas por segundo (fps).

7.1.2 Entorno de Software y Dependencias de Ejecución

El ecosistema digital requiere la presencia de los siguientes intérpretes y servicios activos en segundo plano para habilitar el canal de WebSockets y las APIs multimedia:

- **Subsistema Servidor (Backend):** Entorno de ejecución Python en su versión estable 3.11, aislado mediante un entorno virtual (*venv*) que aloja las librerías OpenCV, MediaPipe, Keras 3 y el servidor de eventos Flask-SocketIO sobre el puerto local 5000.
- **Subsistema Cliente (Frontend):** Servidor de desarrollo en caliente asistido por Node.js (versión 18.0 o superior) y el empaquetador Vite, compatible con navegadores web modernos (Google Chrome o Microsoft Edge) dotados de soporte para la API de reconocimiento *Web Speech API*.
- **Motor de Inferencia Lingüística:** Gestor de servicios Ollama (versión 0.4 o superior) ejecutado de forma nativa como demonio del sistema operativo en el puerto 11434, con los pesos del modelo Llama 3.2 completamente descargados en almacenamiento local.

7.1.3 Condiciones Legales, Privacidad y Licenciamiento

El desarrollo multimedia del proyecto maneja flujos de vídeo en tiempo real procedentes de periféricos locales, lo que exige la consideración del marco normativo vigente:

- **Protección de Datos y Privacidad (RGPD):** Al procesar la extracción de características cinemáticas en local y abstraer de forma inmediata la matriz de píxeles del vídeo en un vector geométrico de 126 flotantes desde el inicio del pipeline, el sistema evita el almacenamiento o la transmisión de imágenes a servidores externos en ningún momento del proceso de reconocimiento. Cabe matizar que, de forma independiente a este flujo visual, la oración final ya traducida sí se transmite de forma cifrada (HTTPS) al servicio en la nube de Google Cloud Text-to-Speech con el único propósito de generar el audio sintetizado, por lo que el cumplimiento del Reglamento General de Protección de Datos exige informar al usuario de este tratamiento puntual de datos textuales antes de su uso.
- **Licenciamiento de Código Abierto:** El código fuente del proyecto se distribuye bajo los términos de la licencia de código abierto MIT, permitiendo su libre modificación, distribución y uso asistivo sin restricciones comerciales. Las librerías de terceros empleadas se rigen por sus respectivas licencias públicas (Apache 2.0 para MediaPipe y TensorFlow, y la licencia permisiva de Meta para el uso de los pesos de Llama 3.2).

7.2 Presupuesto General del Proyecto

El presupuesto económico de este Trabajo de Fin de Grado calcula los costes directos e indirectos asignables al proceso de diseño, entrenamiento del clasificador BiLSTM e implementación de la interfaz web. El ciclo completo de ingeniería multimedia se estima en una dedicación temporal equivalente a 300 horas laborales (correspondientes a la carga de 12 ECTS de la titulación académica).

7.2.1 Costes Directos de Personal

Se considera el coste derivado de las horas de ingeniería computacional dedicadas por el desarrollador en calidad de Ingeniero Multimedia Junior, aplicando una tasa de coste por hora estándar y competitiva en el mercado laboral de desarrollo de software:

Categoría Profesional	Dedicación (Horas)	Coste / Hora (€)	Subtotal (€)
Ingeniero Multimedia Junior (I+D)	300	20,00 €	6.000,00 €
Total Costes de Personal			6.000,00 €

Tabla 7.1: Desglose detallado de los costes directos de personal técnico de ingeniería.

7.2.2 Planificación Temporal del Proyecto

La ejecución del proyecto se ha estructurado en siete fases secuenciales e iterativas a lo largo de los 4 meses de desarrollo, distribuyendo las 300 horas totales de dedicación según la naturaleza y complejidad de cada etapa:

Fase del Proyecto	Dedicación (Horas)	Porcentaje (%)
Investigación y Estado del Arte	35	11,7%
Diseño y Captura del Dataset	30	10,0%
Entrenamiento y Validación del Modelo BiLSTM	45	15,0%
Desarrollo del Backend (Flask, WebSockets, Ollama)	50	16,7%
Desarrollo del Frontend (React, UI/UX)	55	18,3%
Integración, Pruebas y Depuración	35	11,7%
Redacción de la Memoria Técnica	50	16,7%
Total	300	100%

Tabla 7.2: Desglose temporal de las horas de dedicación por fase del proyecto.

Cabe destacar que las fases de desarrollo del *frontend* y del *backend* concentran conjuntamente más del 35% de la dedicación total, reflejando la complejidad de orquestar la comunicación bidireccional en tiempo real entre el cliente web y los múltiples módulos de inteligencia artificial del servidor.

7.2.3 Costes Indirectos: Amortización de Equipamiento

De acuerdo con las directrices de la contabilidad analítica de proyectos, los bienes de equipo físico no se presupuestan por su valor total de compra, sino mediante la amortización lineal aplicada

estrictamente durante los 4 meses que ha durado el ciclo de desarrollo del TFG, asumiendo una jornada estándar de uso:

Equipo / Hardware	Valor de Compra (€)	Vida Útil (Meses)	Uso (Meses)	Amortización (€)
Lenovo IdeaPad 1 Gen 7 15ALC7 (Ryzen 7 5700U)	1.200,00 €	48	4	100,00 €
Webcam USB HD 1080p	60,00 €	24	4	10,00 €
Total Costes de Amortización				110,00 €

Tabla 7.3: Cálculo de la amortización lineal de la infraestructura de hardware empleada.

7.2.4 Costes de Consumo de Servicios de Terceros

El núcleo analítico de inteligencia artificial se ejecuta íntegramente de forma local y gratuita a través del ecosistema de Ollama. No obstante, el subsistema multimedia incorpora una API de pago para la locución neuronal premium de la voz sintetizada, lo que constituye, junto con las herramientas de código abierto empleadas, el desglose completo de licencias y servicios externos consumidos durante el desarrollo:

Concepto / Infraestructura Cloud	Tipo de Licencia	Volumen de Consumo	Coste Total (€)
Entorno de Desarrollo (VS Code, Git)	Código Abierto (MIT)	Ilimitado	0,00 €
Modelos Locales (MediaPipe, Llama 3.2)	Licencia Comunitaria	Ilimitado	0,00 €
API Google Cloud Text-to-Speech	Pago por Uso (SaaS)	3,7 millones de caracteres	15,00 €
Total Costes de Servicios Cloud			15,00 €

Tabla 7.4: Costes de licencias de software y consumo de APIs de infraestructura en la nube.

7.2.5 Resumen del Presupuesto Económico General

La consolidación de los bloques de costes descritos con anterioridad determina la base imponible del presupuesto de desarrollo. Al tratarse de un proyecto técnico ejecutable en el mercado nacional, se le aplica el tipo impositivo ordinario del Impuesto sobre el Valor Añadido (IVA) vigente del 21%:

Bloque de Coste Presupuestado	Importe (€)
Costes Directos de Personal	6.000,00 €
Amortización de Equipamiento Físico	110,00 €
Consumo de Servicios e Infraestructura Cloud	15,00 €
Base Imponible General (Subtotal)	6.125,00 €
Impuesto sobre el Valor Añadido (IVA 21%)	1.286,25 €
PRESUPUESTO TOTAL GENERAL LIQUIDADO	7.411,25 €

Tabla 7.5: Resumen consolidado del presupuesto económico total de ingeniería del proyecto.

En conclusión, el coste total de ejecución del prototipo multimedia asciende a la cantidad de 7.411,25 €, demostrando la viabilidad de diseñar entornos de accesibilidad de alta tecnología bajo presupuestos de ingeniería optimizados.

Capítulo 8

Conclusiones y trabajo futuro

8.1 Conclusiones Generales

La realización de este Trabajo de Fin de Grado ha permitido cumplir con éxito el objetivo principal: el diseño, desarrollo e implementación de un sistema multimedia e inteligente capaz de reconocer y traducir la Lengua de Signos Española (LSE) a español hablado en tiempo real, ejecutando de forma local e independiente de servicios externos la totalidad del pipeline de inteligencia artificial (extracción cinemática, clasificación de gestos y reestructuración lingüística).

A lo largo del proyecto, se ha demostrado la viabilidad de segmentar un pipeline complejo de Inteligencia Artificial en módulos desacoplados y eficientes. La integración de tecnologías como MediaPipe Hands para la extracción geométrica de puntos clave y el clasificador secuencial profundo basado en redes BiLSTM ha demostrado que es posible identificar gestos dinámicos con un coste computacional reducido en comparación con los enfoques tradicionales basados en visión por computador directa sobre vídeo bruto.

Uno de los mayores hitos del desarrollo ha sido la reestructuración del motor lingüístico hacia un enfoque local mediante Ollama y el modelo Llama 3.2. Esta decisión de ingeniería no solo resolvió las limitaciones técnicas asociadas a las tasas de peticiones de las APIs en la nube, sino que dotó al núcleo de procesamiento del sistema (extracción de landmarks, clasificación BiLSTM y reestructuración gramatical) de dos valores fundamentales de cara al panorama tecnológico moderno: la soberanía e intimidad de los datos del usuario al procesar el flujo visual y textual sin salir de la máquina local, y la independencia de la conectividad a internet para todo el razonamiento lingüístico del sistema. Cabe matizar que la etapa final de síntesis de voz (TTS) sí recurre a la API en la nube de *Google Cloud Text-to-Speech* con el fin de garantizar una calidad vocal neuronal homogénea independientemente del sistema operativo del cliente, por lo que dicho módulo concreto requiere de conexión a internet para su funcionamiento.

Finalmente, la consolidación de la interfaz gráfica en React y el canal asíncrono Full-Duplex gestionado por WebSockets confirman que la ingeniería multimedia actual permite crear herramientas de accesibilidad tecnológica de alta fidelidad, estables y preparadas para un despliegue directo en entornos de producción local.

8.2 Obtención de Objetivos

Para evaluar el éxito del proyecto, se repasa el estado de los objetivos específicos planteados al inicio de la investigación:

- **Adquisición y optimización del dataset:** Se ha consolidado una base de datos propia de entrenamiento que cubre las necesidades de un vocabulario de control, optimizado mediante técnicas de aumento de datos sintéticos con ruido gaussiano y desplazamientos aleatorios para robustecer la fase de aprendizaje.
- **Procesamiento temporal de alta precisión:** La red neuronal BiLSTM diseñada ha demostrado una alta capacidad de generalización al asimilar las dependencias temporales de una ventana flotante de 30 fotogramas, logrando clasificar con éxito las 13 clases del sistema bajo un filtro de persistencia estricto.
- **Traducción sintáctica fluida:** Se ha conseguido que un modelo de lenguaje de parámetros optimizados actúe de forma determinista como un intérprete gramatical, convirtiendo glosas inconexas en mayúsculas en oraciones en castellano sintácticamente maduras y correctas.
- **Salida multimedia accesible:** Se ha cerrado el ciclo de comunicación bidireccional al acoplar la síntesis de voz automática inmediatamente después de la recepción de la traducción lingüística, garantizando una experiencia accesible e inclusiva.

8.3 Líneas Futuras

A pesar de los excelentes resultados del sistema actual, la traducción automatizada de lenguas de signos es un campo de investigación abierto y en constante evolución. Se proponen las siguientes líneas de trabajo para la expansión y mejora del prototipo:

- **Ampliación del Vocabulario y Dataset Colectivo:** Incrementar de forma escalada el diccionario de glosas entrenadas mediante la colaboración de múltiples usuarios signantes, lo que aportará una mayor variabilidad al conjunto de datos, mejorando la robustez global del clasificador BiLSTM.
- **Integración de Expresión Facial y Corporal (Mapeo Holístico):** Incorporar los módulos *MediaPipe Face Mesh* y *Pose* para capturar las expresiones no manuales (gestos faciales, posición de los hombros y torso), las cuales desempeñan un papel gramatical y emocional crucial en la comunicación de la LSE y permitirían resolver ambigüedades entre signos con configuraciones manuales idénticas.
- **Optimización de Modelos y Cuantización para Dispositivos Móviles:** Investigar la conversión del modelo BiLSTM a formatos optimizados como TensorFlow Lite y la aplicación de técnicas de cuantización sobre el motor Ollama para permitir la ejecución del sistema completo en dispositivos móviles de recursos limitados sin penalizar el tiempo de respuesta.
- **Comunicación Bidireccional Completa: Traducción de Voz a Signos mediante Avatar 3D:** Se propone la integración de un Avatar 3D virtual (usando tecnologías web como Three.js o Babylon.js) animado en tiempo real. Cuando el usuario oyente hable, un sistema

de reconocimiento de voz capturará el texto y un motor de animación moverá los brazos y manos del avatar en la pantalla para generar los signos de la LSE de vuelta. Eso cerraría el círculo accesible de forma espectacular.

- **Reconocimiento Continuo:** Esto implica investigar arquitecturas basadas en Transformers temporales o pérdidas de tipo CTC (Connectionist Temporal Classification) (el algoritmo que usa Google para los subtítulos automáticos). Esto permitiría al usuario signar de corrido de forma natural, y la IA se encargaría de segmentar las palabras sobre la marcha sin necesidad de buffers rígidos de 30 frames.
- **Migración del Módulo TTS a un Motor de Síntesis de Voz Local:** Sustituir la dependencia actual de la API en la nube de *Google Cloud Text-to-Speech* por un motor de síntesis de voz neuronal ejecutado íntegramente en local. Esta migración eliminaría la última dependencia de conectividad a internet del sistema, completando así la independencia total de los datos del usuario y garantizando un funcionamiento íntegro del pipeline sin necesidad de servicios externos.

Bibliografía

- [1] Roberto Suárez. *Las personas sordas seguimos encontrando barreras que atentan contra nuestra dignidad*. Entrevista a Roberto Suárez, presidente de la CNSE. Revista Perfiles. Consultado en 2025. 2024. URL: <https://revistaperfiles.com/entrevistas/las-personas-sordas-seguimos-encontrando-barreras-que-atentan-contra-nuestra-dignidad/>.
- [2] Jefatura del Estado. *Ley 27/2007, de 23 de octubre, por la que se reconocen las lenguas de signos españolas y se regulan los medios de apoyo a la comunicación oral de las personas sordas, con discapacidad auditiva y sordociegas*. BOE núm. 255, de 24 de octubre de 2007. 2007. URL: <https://www.boe.es/eli/es/l/2007/10/23/27>.
- [3] eseGeCe. *¿Qué son los WebSockets?* eseGeCe Software. URL: <https://www.esegece.com/es/products/websockets/what-are-websockets/> (visitado 08-06-2026).
- [4] Centro Integral de Personas Sordas de Málaga. *Barreras de Comunicación*. Consultado en 2025. URL: <https://cismalaga.es/barreras-de-comunicacion/>.
- [5] Elsevier. *Sign Language Recognition - ScienceDirect - Topics*. 2026. URL: <https://www.sciencedirect.com/topics/computer-science/sign-language-recognition>.
- [6] MathWorks. *Long Short-Term Memory Networks*. Consultado en 2025. 2025. URL: <https://la.mathworks.com/discovery/lstm.html>.
- [7] Sonia Bilbao Arechabala. *De Transformers a agentes de IA: evolución de los LLM, y la inteligencia artificial generativa*. TECNALIA. Consultado en 2026. 2026. URL: <https://www.tecnalia.com/blog/transformers-agentes-ia-evolucion-llm-rag-inteligencia-artificial-generativa>.

Parte II

Anexos

Apéndice A

Manual de Instalación y Configuración del Sistema

Este apéndice describe de forma detallada el protocolo de instalación, configuración y despliegue del sistema de videollamada accesible en un entorno local. Debido a que la arquitectura está compuesta por múltiples piezas de software independientes que interactúan de forma asíncrona, es imperativo seguir el orden secuencial de preparación aquí estipulado para garantizar la correcta inicialización de los canales de comunicación y los motores de inteligencia artificial.

A.1 Requisitos Previos del Sistema

Antes de iniciar la configuración de los repositorios, la máquina anfitriona debe contar con las siguientes herramientas fundamentales instaladas en su sistema operativo:

- **Python (Versión 3.11):** Motor de ejecución del servidor backend y del entorno de inferencia neuronal de TensorFlow y MediaPipe.
- **Node.js (Versión 18.0 o superior) y npm:** Entorno de ejecución y gestor de paquetes necesarios para compilar y levantar la interfaz de usuario basada en Vite y React.
- **Ollama (Versión 0.4 o superior):** Entorno de ejecución local para la gestión y consumo nativo de grandes modelos de lenguaje (LLM).
- **Conectividad a Internet (Solo instalación):** Requerida exclusivamente para la descarga inicial de paquetes de dependencias, bibliotecas y los pesos del modelo lingüístico Llama 3.2.

A.2 Configuración del Entorno de Ejecución Local (Ollama)

El motor lingüístico requiere la inicialización previa del servicio local antes de arrancar el backend de Python. Los pasos para su despliegue son:

1. Descargar e instalar la aplicación oficial de Ollama correspondiente al sistema operativo en uso.
2. Abrir una terminal de comandos del sistema operativo y verificar que el demonio del servicio está activo ejecutando una petición de escucha. El entorno levantará de forma automática una API REST en el puerto local 11434.
3. Ejecutar el siguiente comando en la terminal para descargar de forma persistente los pesos y la configuración del modelo de parámetros optimizados de Meta:

```
ollama pull llama3.2
```

4. Validar que el modelo se encuentra correctamente registrado en el almacenamiento local ejecutando:

```
ollama list
```

A.3 Instalación y Despliegue del Servidor Backend

El núcleo de procesamiento dinámico e inferencia basado en Flask y Socket.IO se gestiona desde la carpeta interna de la arquitectura del repositorio general.

1. Abrir la terminal y navegar hasta la raíz de la carpeta encargada de la lógica del servidor de visión artificial:

```
cd /Sign-Language-Recognition-LSE-/LSE-Sign-Language-Recognition
```

2. Con el propósito de evitar colisiones entre versiones de bibliotecas globales del sistema operativo, se debe inicializar un entorno virtual aislado de Python (*Virtual Environment*):

```
python -m venv venv
```

3. Activar el entorno virtual generado según el sistema operativo de la máquina de desarrollo:

- En entornos Windows (PowerShell): `.\venv\Scripts\Activate.ps1`
- En entornos Linux / macOS: `source venv/bin/activate`

Tras la activación, el prefijo (`venv`) deberá aparecer visible al inicio de la línea de comandos de la terminal.

4. Instalar de forma masiva el árbol de dependencias y bibliotecas críticas (OpenCV, MediaPipe, TensorFlow, Keras y Flask-SocketIO) utilizando el gestor de paquetes e inyectando el archivo de congelación de versiones:

```
pip install -r requirements.txt
```

5. Asegurarse de que el artefacto binario con los pesos óptimos del clasificador de lengua de signos (`bilstm_model.h5`) se encuentra ubicado en la raíz de dicha carpeta.

6. Levantar el servidor de procesamiento concurrente ejecutando el script principal:

```
python server.py
```

El backend quedará en estado de escucha activa, abriendo un canal de comunicación Full-Duplex persistente a través del puerto local 5000.

A.4 Instalación y Despliegue de la Interfaz de Usuario (Frontend)

Una vez que el servidor backend y el motor de Ollama se encuentran operativos en sus respectivos puertos, se procede a compilar la interfaz interactiva de React asistida por Vite.

1. Abrir una nueva pestaña o ventana de la terminal (manteniendo la del backend activa en segundo plano) y desplazarse hasta la raíz del subproyecto de la interfaz:

```
cd /Sign-Language-Recognition-LSE-/Interfaz-LSE/sign-link-app
```

2. Ejecutar el comando de instalación de Node.js para descargar de forma local el árbol de dependencias estructurado en el archivo `package.json`:

```
npm install
```

Este comando generará la carpeta local `node_modules` aislando las bibliotecas de React y el cliente de Socket.IO.

3. Crear un archivo de texto plano denominado exactamente `.env` en la raíz de esta carpeta (al mismo nivel jerárquico que `vite.config.js`). Este archivo es imprescindible, ya que el módulo de síntesis de voz (TTS) del sistema delega la generación del audio en la API en la nube de *Google Cloud Text-to-Speech*, en lugar de en la síntesis nativa del navegador, con el fin de garantizar una calidad de voz neuronal homogénea independientemente del sistema operativo del cliente.
4. Generar una clave de API válida desde la consola de *Google Cloud Platform*, habilitando previamente el servicio *Cloud Text-to-Speech API* sobre el proyecto correspondiente. Abrir a continuación el archivo `.env` e inyectar dicha clave utilizando el prefijo obligatorio de Vite para variables de entorno protegidas:

```
VITE_GOOGLE_TTS_KEY=AIzaSyD_TU_CLAVE_REAL_DE_GOOGLE_CLOUD
```

5. Guardar el archivo y arrancar el servidor de desarrollo en caliente de Vite mediante el script de automatización:

```
npm run dev
```

6. La terminal mostrará una dirección URL local (típicamente `http://localhost:5173`). Abrir dicha ruta en un navegador web convencional (se recomienda Google Chrome o Microsoft Edge por compatibilidad nativa con la *Web Speech API*).

Tras completar este último paso, el entorno multimedia estará completamente interconectado: el cliente web enviará el flujo Base64 al puerto 5000, Flask procesará los landmarks de las manos, Ollama resolverá la gramática en el puerto 11434 y la interfaz web invocará a Google Cloud Text-to-Speech para verbalizar el resultado por voz, cerrando el ciclo accesible del TFG.